Practical Silent Threshold Signatures and Silent Threshold Encryption for Dynamic Committees*

Yifei He
School of Computer Science
Peking University
Beijing, Beijing, China
heyifei@pku.edu.cn

Zheng Zhou
School of Computer Science
Peking University
Beijing, Beijing, China
zhengzhoum@stu.pku.edu.cn

Yu Chen†‡
School of Cyber Science and Technology
Shandong University
Qingdao, Shandong, China
yuchen.prc@gmail.com

Zhi Guan
National Engineering Research Center For Software Engineering
Peking University
Beijing, Beijing, China
guan@pku.edu.cn

Zhong Chen§
School of Computer Science
Peking University
Beijing, Beijing, China
zhongchen@pku.edu.cn

ABSTRACT

Silent threshold signatures (STS) and encryption (STE) enable threshold cryptography without interactive distributed key generation, allowing a group of N parties to non-interactively generate a joint public signature verification key or an encryption key. However, modern distributed systems (such as Ethereum) rely on small, dynamically changing committees of size $n \ll N$ for efficiency, and existing silent threshold schemes either fail to support this dynamic setting or suffer from severe scalability issues. To the best of our knowledge, the only prior STS construction for dynamic committees, Dyna-hinTS, requires an aggregation time of $O(N \log N)$ per epoch, tightly coupling the cost to the global system size rather than the small active committee. Furthermore, no STE scheme for dynamic committees has been proposed yet.

In this work, we present practical silent threshold signature and encryption schemes for dynamic committees, reducing the aggregation cost so that it depends only on the committee size n. For signatures, we redesign the Dyna-hinTS framework by replacing its Plonk-style SNARKs with linear pairing checks and a new polynomial commitment for representing the committee, yielding an aggregation time of $O(n \log^2 n)$. We also introduce the first silent threshold encryption scheme for dynamic committees with matching efficiency. We further significantly optimize the silent setup phase common to prior STS and STE schemes, reducing each party's one-time setup (i.e., generating the setup data, referred to as a "hint") cost from $O(N^2)$ to $O(N)$.

We implement our schemes in Rust, and the results demonstrate practicality at scale. For a system parameterized with $N = 2^{20}$ and $n = 2^{10}$, the per-party hint generation takes 197 seconds, and signature aggregation takes 0.153 seconds, achieving a $> 1900\times$ improvement over Dyna-hinTS. At the same time, our aggregated signature size, verification key size, and verification time (with access to the epoch committee representation) remain constant.

CCS CONCEPTS

• **Security and privacy** → **Digital signatures**; **Public key encryption**.

KEYWORDS

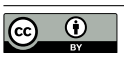
Threshold Cryptography, Silent Threshold Signature, Silent Threshold Encryption, Dynamic Committees, Non-interactive Setup

1 INTRODUCTION

Threshold signatures and threshold encryption are important primitives in modern cryptography. Given a set of N parties and a threshold $t \leq N$, any subset of at least t parties can jointly sign a message or decrypt a ciphertext, while fewer than t parties cannot forge signatures or learn any information from the ciphertext. Traditionally, constructing such threshold cryptography schemes relies on distributed key generation (DKG) as the setup phase. This process requires interaction among all parties to jointly generate and share a single global secret key, and to publish the corresponding public key for signature verification or encryption.

However, the interactive nature of DKG incurs high communication and computational overhead, and typically assumes a synchronous setting. Moreover, jointly generating the shared key makes it difficult to dynamically adjust the threshold or the set of parties, and these limitations become increasingly severe as the system scales up. This has motivated recent works [9, 19, 26, 27, 34] on *silent* threshold cryptography, which replaces DKG with a non-interactive setup. Garg et al. [26, 27] introduced silent threshold signature (STS) and encryption (STE), where each party provides an additional "hint" that implicitly encodes information about its private key, and the aggregator can use these "hints" to generate a

*This is the full version of the paper accepted to the 2026 ACM Conference on Computer and Communications Security (CCS '26, Cycle B).
†Also with, State Key Laboratory of Cryptography and Digital Economy SecurityShandong University.
‡Also with State Key Laboratory of Cryptology.
§Also with, School of AI and Liberal ArtsBeijing Normal-Hong Kong Baptist University.



This work is licensed under a Creative Commons Attribution 4.0 International License.

succinct proof, demonstrating the validity of the aggregate signature or performing decryption on the ciphertext. These schemes also supported dynamic threshold adjustment and achieved constant-size verification keys, signatures, and verification time.

On the other hand, large-scale distributed systems [3–5, 18, 28] are gradually shifting to **dynamic committee settings** for efficiency and security considerations. In such a setting, the system operates in recurring epochs, and only a randomly chosen small subset of parties Com_e of size $n \ll N$ is authorized to sign or decrypt in each epoch e. This significantly reduces the cost without sacrificing security, because t and n can be much smaller than N to ensure that an adversary controlling f parties has negligible probability of controlling $\geq t$ parties within a randomly sampled committee.

However, existing silent threshold schemes described above cannot restrict authorization to any specific subset of parties: their silent setup is performed globally by all parties, which can only guarantee that $\geq t$ of all N (instead of n) parties generated the signature or decrypted the ciphertext. To address this problem, Kate et al. [29] further proposed the notion of silent threshold signature for dynamic committees (c-STS) and a construction called Dyna-hinTS, which shared a common construction with [26]. Despite its conceptual breakthrough, the signature aggregation of Dyna-hinTS suffers from an $O(N \log N)$ computational cost: to additionally prove that all partial signatures used for aggregation are from the required committee, the aggregator must interpolate and evaluate dense global state polynomials of degree $O(N)$ to generate several Plonk-style proofs. This forces the aggregation cost to scale with the massive global system rather than the small active committee, which may become a bottleneck in settings such as Ethereum, where $N \approx 2^{20}, n \approx 2^{10}$ and the committee changes every 12 seconds. Intuitively, if signing or encryption involves only a small committee, the associated overhead should depend only on the size of this committee rather than the size of the entire system. In consequence, we ask the following question:

Can we construct a silent threshold signature scheme for dynamic committees whose aggregation cost only depends on the size of the committee?

Furthermore, we observe that silent threshold signature and encryption schemes share similar structures, but only the former has been extended to support dynamic committees. Considering that threshold encryption schemes also have unique applications in decentralized systems, bridging this gap is highly desirable. A typical use case is the encrypted mempool, where users can encrypt their transactions before submitting them to mitigate Miner Extractable Value (MEV), and these transactions can be threshold-decrypted by consensus nodes only after their order is fixed. To deploy such a mechanism on large blockchains such as Ethereum, the decryption authority must be restricted to the specific committee of the current epoch, which changes dynamically. Therefore, a practical STE scheme for dynamic committees could be an important building block toward encrypted mempools. This leads to our second question:

Can we construct a silent threshold encryption scheme for dynamic committees?

In this work, we answer both of them in the affirmative: we demonstrate how to build a practical silent threshold signature and encryption scheme for dynamic committees based on existing techniques [26, 27, 29], with aggregation cost quasilinear in the committee size n and independent of the system size N.

1.1 Our Contributions

Practical STS scheme for dynamic committees. We propose a practical STS scheme for dynamic committees, whose signature aggregation time dynamically adapts to each epoch and scales only quasilinearly in the corresponding committee size n (rather than the system size N). Inspired by the degree-check techniques from [27], we redesign the polynomial commitment representing the epoch committee to strictly enforce that the signers form a subset of this actively rotating committee, which allows us to remove the Plonk-style proofs in [29] and prove the subset relation purely through polynomial divisibility, yielding fully linear verification equations over pairing groups. Furthermore, to ensure the system remains oblivious to the system size N even as the committee changes, we mathematically bypass full-domain polynomial interpolation in signature aggregation and propose a method to compute all required group elements strictly over the dynamically updated subset of roots of unity (corresponding to the epoch committee). As a result, the complexity of signature aggregation is decoupled from the global setup, reducing from $O(N \log N)$ (which was due to generating the Plonk-style proofs) to $O(n \log^2 n)$, making our scheme suitable for much larger systems with dynamic committees.

Silent threshold encryption scheme for dynamic committees. We introduce the notion of silent threshold encryption for dynamic committees (c-STE), extending STE to support dynamic committee settings in which the decryption authority rotates to a specific committee for epoch e. During the one-time silent setup phase, each party generates a public/secret key pair independently and a succinct hint that contributes to a compact encryption key. When the system transitions to a new epoch e, only a subset Com_e (with size n and threshold t) of all parties is authorized for decryption without any re-setup. Messages encrypted to epoch e can be decrypted by any $\geq t$ parties in Com_e in a non-interactive manner. The security of c-STE guarantees that an adversary cannot distinguish the ciphertext from a random group element as long as $< t$ parties in Com_e are corrupted (there is no limitation on the adversary for corrupting parties beyond Com_e). Furthermore, we present a concrete c-STE construction that inherits the same efficiency properties as our c-STS scheme: the computational overhead strictly scales with the committee size n, making it highly practical for large-scale systems.

Optimized silent setup. We significantly improve the setup phase of silent threshold cryptography schemes based on the idea proposed in [26], by reducing the per-party hint generation complexity from $O(N^2)$ to $O(N)$. To compute $O(N)$ hint elements, existing schemes natively compute multi-scalar multiplications (MSMs) of size $O(N)$ for each of them. In comparison, we exploit the algebraic properties of Lagrange basis polynomials over roots of unity and demonstrate how to mathematically express these complex quotient polynomials as sparse linear combinations of precomputed base commitments, which allows us to compute each element using

only $O(1)$ group operations on average. This optimization can be applied to [26, 27, 29] and this work since all of these schemes share a common structure and the same setup phase. Considering that the related technique does not correspond to any specific silent threshold signature or encryption schemes, this part may be of independent interest.

Implementation and evaluation. We implement our schemes in Rust and evaluate them on realistic system scales. For a system parameterized with $N = 2^{20}$ and $n = 2^{10}$, generating a single hint takes only 197.2 seconds, while the naive approach used in prior schemes becomes infeasible. Signature aggregation takes only about 0.153 seconds, yielding a $> 1900\times$ improvement over Dyna-hinTS. Signing and verification (with access to the epoch committee representation) remain constant time (<1ms and about 10ms, respectively), with a constant-size aggregated signature and verification key. We also analyze and estimate the cost of the one-time preprocessing, demonstrating its practicality in large-scale systems.

1.2 Related Work

Silent setup for threshold cryptography. There are two main approaches to constructing a silent threshold signature (STS) scheme. The first delegates the public key aggregation to the signature verifier. A typical example is the BLS multi-signature [9, 13], in which the aggregator must provide the list of signers as part of the aggregated signature, and the verifier aggregates all corresponding public keys. As a result, both the signature size and verification cost extend linearly with the number of signers, leading to high communication and storage overhead. While recent works optimize either signature size [6, 7] or verification key size [34], they fail to minimize both simultaneously, resulting in massive signatures.

The second approach delegates the public key aggregation to the aggregator. This yields succinct aggregated signatures, but the aggregator must additionally provide a correctness proof. Succinct non-interactive arguments (SNARKs) naturally meet this requirement, as their succinctness and efficiency greatly reduce communication costs and verification overhead. However, the aggregation cost and parameter choices in such schemes are restricted by the underlying proof system. Building on this approach, several STS schemes [19, 26] design customized SNARKs to reduce aggregation costs further. A typical example is hinTS [26], which represents the set of signers as a binary vector. The aggregator commits to this vector and uses a set of customized succinct Plonk [23]-style proofs to ensure that the number of signers meets the threshold. The aggregated public key is computed as the inner product between this vector and the list of public keys, and its correctness is guaranteed using an inner-product argument called generalized sumcheck [36]. With these techniques, hinTS achieved a constant signature size, verification key size, and verification complexity.

Constructing silent threshold encryption (STE) schemes is significantly more challenging, and no practical solutions were known for a long time. A closely related notion is distributed broadcast encryption, which can be viewed as a special case of STE with $t = 1$. However, even this special case has only feasible constructions [16, 17, 21, 31, 39] in the past several years. Before the work of Garg et al. [27], it was widely believed that building STE necessarily required heavy cryptographic primitives such as indistinguishability Obfuscation [10] (e.g., [37]) or universal witness encryption [24, 25]. The work of Garg et al. [27] first proposed the notion of STE and provided a concretely efficient construction based on pairing-friendly groups. Specifically, they proposed a paradigm that derives signature-based witness encryption from signature schemes with linear verification, and applied this paradigm to a linear-verification variant of hinTS [26] to obtain the corresponding STE scheme.

Threshold cryptography schemes for dynamic committees. Since traditional DKGs do not support changes to the committee, prior work [11, 20, 32, 38, 40] has long focused on dynamic secret sharing to change parties while keeping the secret value (public key) fixed. However, this incurs higher complexity and tolerates lower adversarial thresholds. Recently, Baird et al. [8] proposed Multiverse Threshold Signature (MTS), enabling arbitrary universes (subsets) of signers to hold distinct thresholds. Although parties hold constant-size keys and the size of partial signatures is independent of the number of universes, the signers must interact among themselves and the aggregator to set up the universe's verification key when they are assigned to a particular universe. This interaction was eliminated by the STS scheme [26] via a one-time global setup.

Building on this, Kate et al. [29] proposed Dyna-hinTS to support dynamic committees. Dyna-hinTS allows the committee to change per epoch using the same global verification key from a one-time setup. It yields an improved MTS scheme that bypasses any per-universe setup requirement, though its aggregation efficiency remains constrained by the global system size.

2 PRELIMINARIES

Notions. We use λ for the security parameter, and $\mathsf{negl}(\lambda)$ for the negligible function, i.e., for any polynomial f, there exists a λ_0 such that for all $\lambda > \lambda_0, \mathsf{negl}(\lambda) < 1/f(\lambda)$. We use $[n]$ for the set $\{1, 2, \ldots, n\}$. For a set S, we use S' for $S \cup \{0\}$, and use $[n]'$ for $[n] \cup \{0\} = \{0, 1, \ldots, n\}$.

2.1 Bilinear Groups

We use $(\mathbb{Z}_p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, g_1, g_2, g_T)$ for a pairing-friendly group, where $\mathbb{Z}_p$ is a prime field of order $p = \exp(\Omega(\lambda))$, and $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are groups of order p with a non-degenerate pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$. $g_1 \in \mathbb{G}_1, g_2 \in \mathbb{G}_2$ are uniformly sampled generators such that $e(g_1, g_2) = g_T$. For $x \in \mathbb{Z}_p$, we use $[x]_s$ to denote $x \cdot g_s$, and we denote the group operation additively, namely $[x]_s + [y]_s = [x+y]_s$ for $s \in \{1, 2, T\}$. We denote the pairing operation $e([x]_1, [y]_2)$ by $[x]_1 \circ [y]_2 = [y]_2 \circ [x]_1 = [x \cdot y]_T$. We also use this notion for the matrix of group elements, i.e., for a $k_1 \times k_2$ matrix $\boldsymbol{A} = \{[a_{i,k}]_{\alpha_k}\}$ and a $k_2 \times k_3$ matrix $\boldsymbol{B} = \{[b_{k,j}]_{\beta_k}\}$, if $\alpha_k + \beta_k = 3$ for all $k = 1, 2, \ldots, k_2$, then $\boldsymbol{A} \circ \boldsymbol{B}$ is well-defined, and its result is a $k_1 \times k_3$ matrix $\boldsymbol{C} = \{[c_{i,j}]_T\}$ that satisfies $[c_{i,j}]_T = \sum_{k=1}^{k_2} [a_{i,k}]_{\alpha_k} \circ [b_{k,j}]_{\beta_k}$. It is obvious that $c_{i,j} = \sum_{k=1}^{k_2} a_{i,k} b_{k,j}$.

2.2 BLS Signature

Definition 2.1 (BLS Signature [14]). Let $\mathcal{H} : \{0,1\}^* \to \mathbb{G}_2$ be a random oracle. The BLS signature consists of the following algorithms:

- BLS.Gen: It samples a random $\mathsf{sk} \leftarrow \mathbb{Z}_p^*$ and outputs a public/secret key pair as $(\mathsf{pk} = [\mathsf{sk}]_1, \mathsf{sk})$.
- BLS.Sign$(\mathsf{sk}, \mathsf{msg})$: It outputs a signature $\sigma = \mathsf{sk} \cdot \mathcal{H}(\mathsf{msg}) \in \mathbb{G}_2$.
- BLS.Verify$(\mathsf{pk}, \mathsf{msg}, \sigma)$: It verifies the validity of the signature by checking $\mathsf{pk} \circ \mathcal{H}(\mathsf{msg}) \stackrel{?}{=} [1]_1 \circ \sigma$.

2.3 Polynomials over $\mathbb{Z}_p$

Let $\omega \in \mathbb{Z}_p^*$ be an M-th primitive root of unity, and the multiplicative subgroup of roots of unity generated by ω is denoted by $\mathbb{H} = \{\omega, \omega^2, \ldots, \omega^M\}$. Let $L_i(X)$ be the i-th Lagrange basis polynomial on $\mathbb{H}$, namely the unique degree $M-1$ polynomial which satisfies $L_i(\omega^j)$ is 1 when $i = j$ and 0 when $i \neq j$. Let $Z(X) = \prod_{i \in [M]} (X - \omega^i)$ be the vanishing polynomial on $\mathbb{H}$. Since $\mathbb{H}$ is a multiplicative subgroup, we have $Z(X) = X^M - 1$, and $L_i(X) = \frac{\omega^i}{M} \cdot \frac{X^M - 1}{X - \omega^i}$ with $L_i(0) = \frac{1}{M}$ for $i \in [M]$. Sometimes we also write ω^M as ω^0 and L_M as L_0 since they are equivalent.

For any vector $B = (b_1, b_2, \ldots, b_M)$[1], let $B(X) = \sum_{i \in [M]} b_i \cdot L_i(X)$ as the unique polynomial that satisfies $\deg(B) \leq M-1$ and $B(\omega^i) = b_i$ for all $i \in [M]$.

2.4 Polynomial Commitment

We use the KZG polynomial commitment [30] as the vector commitment scheme. Given a degree bound d, the common reference string derived by a trapdoor τ is denoted by $\mathsf{CRS} = ([1], [\tau], [\tau^2], \ldots, [\tau^d])$. For any polynomial $B(X)$ that satisfies $\deg(B) \leq d$, its commitment is $[B(\tau)]$.

2.5 Generalized Sumcheck

Our construction relies on the following lemma, known as generalized sumcheck, whose proof can be found in [36].

Lemma 2.2 (Generalized Sumcheck). *Let* $A(X) = \sum_{i=1}^{|\mathbb{H}|} a_i \cdot L_i(X), B(X) = \sum_{i=1}^{|\mathbb{H}|} b_i \cdot L_i(X)$, *then*

$$A(X) \cdot B(X) = \frac{\sum_i a_i b_i}{|\mathbb{H}|} + Q_x(X) \cdot X + Q_Z(X) \cdot Z(X)$$

where Q_x, Q_Z *are polynomials with degree* $\leq |\mathbb{H}| - 2$ *defined as*

$$Q_x(X) = \sum_i a_i b_i \frac{L_i(X) - L_i(0)}{X}$$

$$Q_Z(X) = \sum_i a_i b_i \frac{L_i^2(X) - L_i(X)}{Z(X)} + \sum_{i \neq j} a_i b_j \frac{L_i(X) L_j(X)}{Z(X)}$$

3 REVISITING DEFINITION OF C-STS

To better demonstrate our construction, we first revisit the definition of c-STS in [29] and introduce several adjustments in this section. These adjustments include:

- The maximum number of parties in the system N is added as an input to the Setup algorithm since it determines the size of CRS.
- We replace the ComSelect algorithm from [29] with a new abstraction, EpochRepr, to decouple our cryptographic construction from the underlying committee election mechanism. Instead of relying on a trusted third party, we define EpochRepr as a deterministic, publicly computable function that extracts the mathematical representation of the committee (e.g., the polynomial commitment $[B_{\mathsf{Com}_e}(\tau)]_2$) directly from the agreed-upon on-chain state at the start of epoch e. We assume the underlying blockchain (e.g., via a randomness beacon [29]) securely handles this election to guarantee at least t honest parties, and vulnerabilities in the selection mechanism itself are orthogonal to our cryptographic design. Under this abstraction, the representation output by EpochRepr is inherently consistent and publicly available to Verify (and Enc of Definition 7.1) without additional cost or trust assumptions. The related discussion can be found in Section 6.3.
- We add additional efficiency requirements to EpochRepr and Aggr. Considering the important role of EpochRepr, it is natural to set the formal requirement, while the latter is based on our goal of constructing practical schemes as described in Section 1.

Definition 3.1 (c-STS, revisited). A silent threshold signature scheme for dynamic committees consists of a tuple of algorithms $\Sigma = (\mathsf{Setup}, \mathsf{KeyGen}, \mathsf{HintGen}, \mathsf{Preprocess}, \mathsf{EpochRepr}, \mathsf{Sign}, \mathsf{PartVerify}, \mathsf{Aggr}, \mathsf{Verify})$ with the following syntax:

- $\mathsf{CRS} \leftarrow \mathsf{Setup}(1^\lambda, N)$: On input the security parameter λ and an upper bound N on the maximum number of users, the Setup algorithm outputs a common reference string CRS.
- $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$: On input the security parameter λ, the KeyGen algorithm outputs a key pair $(\mathsf{pk}, \mathsf{sk})$.
- $\mathsf{hint}_i \leftarrow \mathsf{HintGen}(\mathsf{CRS}, sk, N, i)$: On input the CRS, the secret key sk, the number of parties N, and a position $i \in [N]$, the HintGen algorithm outputs a hint hint_i.
- $(\mathsf{ak}, \mathsf{vk}) \leftarrow \mathsf{Preprocess}(\mathsf{CRS}, \{\mathsf{hint}_i, pk_i\}_{i \in [N]})$: On input the CRS, all pairs $\{\mathsf{hint}_i, pk_i\}_{i \in [N]}$, the Preprocess algorithm outputs an aggregation key ak and a verification key vk.
- $B_e \leftarrow \mathsf{EpochRepr}(e)$: On input epoch e, the EpochRepr algorithm outputs a mathematical representation B_e of the epoch e's committee $\mathsf{Com}_e \subseteq [N]$.
- $\sigma_i \leftarrow \mathsf{Sign}(\mathsf{sk}_i, m, e)$: On input a secret key sk_i, a message m, and epoch e, the Sign algorithm outputs a partial signature σ_i.
- $1/0 \leftarrow \mathsf{PartVerify}(\mathsf{pk}_i, \sigma_i, m, e)$: On input a message m, a partial signature σ_i, a public key pk_i, and epoch e, the PartVerify algorithm outputs 1 iff the partial signature verifies and corresponds to epoch e.
- $\sigma \leftarrow \mathsf{Aggr}(\mathsf{CRS}, \mathsf{ak}, m, \{\sigma_i\}_{i \in S}, t, e)$: On input the CRS, the aggregation key ak, a message m, a set of partial signatures $\{\sigma_i\}_{i \in S}$, a threshold t, and epoch e, the Aggr algorithm outputs an aggregated signature σ.

[1] If B has length $l_B < M$, we consider that $b_i = 0$ for all $i = l_B + 1, \ldots, M$.

- $1/0 \leftarrow \mathsf{Verify}(\mathsf{vk}, m, \sigma, t, e)$: On input the verification key vk, a message m, an aggregated signature σ, a threshold t, and epoch e, the Verify algorithm outputs 1 iff σ is verified.

Moreover, the c-STS scheme must have the following efficiency requirements:

- The size of the verification key vk and the aggregated signature σ should be $O(1)$.
- For epoch e with the committee size $n = |\mathsf{Com}_e|$, the complexity of EpochRepr should be $O(n \cdot \mathrm{polylog}(n))$.
- Given the commitment of a committee B_e, the complexity of Verify should be $O(1)$, and the complexity of Aggr should be $O(n \cdot \mathrm{polylog}(n))$.

Based on the modification above, we also revisit the correctness and unforgeability properties of c-STS. We defer the formal definition to Appendix A.

4 LINEAR VERIFIABLE C-STS SCHEME

In this section, we first recall the c-STS scheme Dyna-hinTS proposed in [29], and discuss how to use the linear verification equations to replace the Plonk-style weight proofs.

4.1 Revisiting Dyna-hinTS

Dyna-hinTS is a silent threshold signature scheme for dynamic committees based on hinTS [26], but the authorized committee is a subset of the entire set of parties that changes dynamically over epochs. Assume there are N parties in the system and $|\mathbb{H}| = N + 1$. During the silent setup, each party will sample an independent BLS key pair $\{\mathsf{pk}_i = [\mathsf{sk}_i]_1, \mathsf{sk}_i\}$. At epoch e, the authorized committee Com_e is randomly chosen[2]. Given a message m to sign, parties compute an epoch-specified[3] $\mathsf{tag} = \mathcal{H}(m, e)$ and sign it using the BLS signature by $\sigma_i = \mathsf{sk}_i \cdot \mathsf{tag}$. Suppose a subset $B \subseteq [N]$ of parties have signed tag; the aggregator will proceed to aggregate the partial signatures as follows. It first shall aggregate the partial signatures $\{\sigma_i\}_{i\in B}$ and public keys $\{\mathsf{pk}_i\}_{i\in B}$ as σ^* and aPK. Furthermore, to prove the honest aggregation of aPK, the aggregator should commit to B and perform the following computation:

1. It computes proof $\pi_1 = ([Q_Z(\tau)]_1, [Q_x(\tau)]_1, [\hat{Q}_x(\tau)]_1)$ which relies on the generalized sumcheck protocol to prove that the inner-product of public keys and B is the claimed aggregate public key aPK:
$$[SK(\tau)]_1 \circ [B(\tau)]_2 = [1]_2 \circ \mathsf{aPK} + [Z(\tau)]_2 \circ [Q_Z(\tau)]_1 + [\tau]_2 \circ [Q_x(\tau)]_1$$
$$[\tau]_2 \circ [Q_x(\tau)]_1 = [1]_2 \circ [\hat{Q}_x(\tau)]_1$$
where $SK(X) = \sum_{i\in[N]} \mathsf{sk}_i \cdot L_i(X)$ and $B(X) = \sum_{i\in[N]} b_i \cdot L_i(X)$. The first equation checks that the polynomial identity holds at the random point $x = \tau$, and the second equation constrains the maximum degree of $Q_x(X)$ as required in the sumcheck protocol.
2. It computes a Plonk-style proof π_2 to demonstrate that (1) B is a binary vector, (2) the inner product between B and $(1, 1, \ldots, 1)$ equals t, and (3) B is a subset of the authorized committee Com_e. In particular, (3) is achieved in the same way as (1) and (2): By proving that $\bar{B} = \mathsf{Com}_e \setminus B$ is also a binary vector and the inner product between $\bar{B}$ and $(1, 1, \ldots, 1)$ equals $|\mathsf{Com}_e| - t$, it is sufficient to show that $B \subseteq \mathsf{Com}_e$ (notice that given the commitment $[B_{\mathsf{Com}_e}(\tau)]_2$ of Com_e, we have $[\bar{B}(\tau)]_2 = [B_{\mathsf{Com}_e}(\tau)]_2 - [B(\tau)]_2$).

The final signature is $(\sigma^*, \mathsf{aPK}, [B(\tau)]_2, \pi_1, \pi_2)$. For verification, one verifies π_1, π_2 and verifies the aggregated signature σ^* by checking $\mathsf{tag} \circ \mathsf{aPK} = [1]_1 \circ \sigma$.

4.2 Removing Plonk-style Proofs

Since the computation of $[B(\tau)]_2, \mathsf{aPK}, [Q_Z(\tau)]_1, [Q_x(\tau)]_1, [\hat{Q}_x(\tau)]_1$ only relates to $i \in B$, it has complexity $O(n)$. However, generating Plonk-style proofs for proving B and $\bar{B}$ have sufficient weight requires $O(N \log N)$ (below we use B, and the same also applies for $\bar{B}$) because it contains the following steps (slightly adapted and rephrased here for consistency with our notation; note that $L_{N+1} = L_0$):

- Compute the partial sum vector $parsum$ defined by
$$parsum_1 = 0$$
$$parsum_{i+1} = parsum_i + b_i \qquad i = 1, \ldots, N,$$
the corresponding polynomial
$$Parsum(X) = \sum_{i=1}^{N+1} parsum_i \cdot L_i(X)$$
and its commitment $[Parsum(\tau)]_1$.
- To prove the correctness of $[Parsum(\tau)]_1$ and $[B(\tau)]_2$, i.e.,
$$\begin{aligned} Parsum(X) = 0 \qquad & X = \omega \\ Parsum(\omega X) - Parsum(X) - B(X) = 0 \qquad & X \in \mathbb{H} \setminus \{\omega^{N+1}\} \\ Parsum(X) = t \qquad & X = \omega^{N+1} \\ B(X) \cdot (1 - B(X)) = 0 \qquad & X \in \mathbb{H}, \end{aligned}$$
compute the corresponding quotient polynomial $Q_i(X)$ for $i = 1, 2, 3, 4$ where
$$\begin{aligned} L_1(X) \cdot Parsum(X) &= Q_1(X) \cdot Z(X) \\ (X - \omega^{N+1})(Parsum(\omega X) - Parsum(X) - B(X)) &= Q_2(X) \cdot Z(X) \\ L_{N+1}(X) \cdot (Parsum(X) - t) &= Q_3(X) \cdot Z(X) \\ B(X) \cdot (1 - B(X)) &= Q_4(X) \cdot Z(X) \end{aligned}$$
and their (linearly combined) commitment
$$[Q(\tau)]_1 = \Big[\sum_{i=1}^{4} Q_i(\tau) \cdot v^{i-1} \Big]_1$$
for some random challenge v.
- For some random challenge r, compute $B(r), Q(r), Parsum(r), Parsum(\omega r)$ with the corresponding KZG opening proofs.

Therefore, to reduce the complexity of aggregating signatures, we must avoid generating these proofs and use alternative methods to provide a proof of weight. [27] proposed a tricky way to achieve this goal: They compute $B(X)$ by interpolating $\{(i, 0)\}_{[N]\setminus B}$, and a valid B will satisfy $\deg(B) \le N - t$ which can be checked linearly via $[\tau^t]_1 \circ [B(\tau)]_2 = [1]_2 \circ [\hat{B}(\tau)]_1$ with an additional commitment of $\hat{B}(X) = B(X) \cdot X^t$. This works because $\mathsf{aPK} = \sum_{i=1}^{n} \mathsf{pk}_i \cdot b_i$ is an

[2]Here we treat Com_e as a binary vector $\mathsf{Com}_e = \{b_1, \ldots, b_N\} \in \{0, 1\}^N$ where $b_i = 1$ if and only if $i \in \mathsf{Com}_e$. The same applies to B and $\bar{B}$, which we describe later.
[3]Embedding epoch e into the signature prevents the signature from being used for other epochs.

aggregation of sufficiently many ($\geq t$) public keys as long as this vector B has $\geq t$ non-zero coordinates, even if B is not binary. By the security of the BLS multi-signature, it is unforgeable if one does not have $\geq t$ partial signatures.

However, this trick cannot be used directly to replace π_2: π_2 proves that the weight of B (resp. $\bar{B}$) **equals** t (resp. $n-t$), and the degree check is limited to proving the weight $\geq t$. Even if it is used for proving the weight of $B \geq t$, the corresponding statement for $\bar{B}$ becomes "the weight of $\bar{B} \leq n-t$" (not $\geq$), which is also beyond the ability of the degree check.

Our key observation for addressing this technical challenge is that **the vector Com_e does not have to be binary either**. One possible approach is to format $B_{\mathsf{Com}_e}(X)$ the same as $B(X)$, i.e., to compute $B_{\mathsf{Com}_e}(X)$ by interpolating $\{(i,0)\}_{[N]\setminus\mathsf{Com}_e}$. In this way, "$B \subseteq \mathsf{Com}_e$" is equivalent to "the zeros of $B_{\mathsf{Com}_e}(X)$ is a subset of the zeros of $B(X)$", and is further equivalent to the existence of the quotient polynomial $\bar{B}(X) = \frac{B(X)}{B_{\mathsf{Com}_e}(X)}$. Therefore, we obtain the following equation for the subset check:

$$[1]_1 \circ [B(\tau)]_2 = [B_{\mathsf{Com}_e}(\tau)]_2 \circ [\bar{B}(\tau)]_1$$

The aggregator can prove $B \subseteq \mathsf{Com}_e$ by providing the commitment $[\bar{B}(\tau)]_1$ of the quotient polynomial. This also allows us to verify the weight of B via the degree check, where the aggregator should provide the degree proof $[\hat{B}(\tau)]_1$:

$$[\tau^t]_1 \circ [B(\tau)]_2 = [1]_2 \circ [\hat{B}(\tau)]_1 \tag{1}$$

Finally, to ensure that $B(X)$ is non-zero, a dummy party P_0 is always required in B (as was done in [27]), which can also be checked by a KZG opening equation

$$[1]_1 \circ \boxed{[B(\tau)]_2} = [\tau - 1]_2 \circ \boxed{[Q_0(\tau)]_1} + [1]_T.$$

Notice that the minimum weight of B should be $t+1$ after introducing P_0 to B, so we have $\hat{B}(X) = X^{t+1}B(X)$, and the first term of Equation 1 should also be raised to $[\tau^{t+1}]_1$.

So far, we have replaced the Plonk-style weight proofs by introducing a new representation of Com_e and new polynomials with their commitments. Now the aggregated signature $\boldsymbol{w}$ consists of

$$[B(\tau)]_2, \mathsf{aPK}, [Q_Z(\tau)]_1, [Q_x(\tau)]_1, [\hat{Q}_x(\tau)]_1, \sigma^*, [\hat{B}(\tau)]_1, [\bar{B}(\tau)]_1, [Q_0(\tau)]_1$$

which should satisfy the verification that consists of the following six pairing equations (the group elements from the signature are shown with shadow, and the rest are public group elements):

$$\begin{aligned}
[SK(\tau)]_1 \circ \boxed{[B(\tau)]_2} &= [1]_2 \circ \boxed{\mathsf{aPK}} + [Z(\tau)]_2 \circ \boxed{[Q_Z(\tau)]_1} \\
&\quad + [\tau]_2 \circ \boxed{[Q_x(\tau)]_1} \\
[\tau]_2 \circ \boxed{[Q_x(\tau)]_1} &= [1]_2 \circ \boxed{[\hat{Q}_x(\tau)]_1} \\
\mathsf{tag} \circ \boxed{\mathsf{aPK}} &= [1]_1 \circ \boxed{\sigma^*} \\
[\tau^{t+1}]_1 \circ \boxed{[B(\tau)]_2} &= [1]_2 \circ \boxed{[\hat{B}(\tau)]_1} \\
[1]_1 \circ \boxed{[B(\tau)]_2} &= [B_{\mathsf{Com}_e}(\tau)]_2 \circ \boxed{[\bar{B}(\tau)]_1} \\
[1]_1 \circ \boxed{[B(\tau)]_2} &= [\tau - 1]_2 \circ \boxed{[Q_0(\tau)]_1} + [1]_T
\end{aligned} \tag{2}$$

4.3 Our Construction

Now we give our construction based on the above discussion. Let N be the maximum size of the entire set of parties. Without loss of generality, we set $|\mathbb{H}| = N+1$ where the index 0 is reserved for a dummy party with $\mathsf{sk}_0 = 1$ and $\mathsf{pk}_0 = [1]_1$.

Construction 1 (Linear verifiable c-STS). *We specify each algorithm of linear verifiable c-STS as follows.*

- $\mathsf{Setup}(1^\lambda, N)$: *Sample* $\tau \leftarrow_\$ \mathbb{Z}_p$ *and output*

$$\mathsf{CRS} = ([\tau^1]_1, \ldots, [\tau^{N+1}]_1, [\tau^1]_2, \ldots, [\tau^{N+1}]_2).$$

- $\mathsf{KeyGen}(1^\lambda)$: *Sample* $x \leftarrow_\$ \mathbb{Z}_p^*$ *and output* $\mathsf{pk} = [x]_1, \mathsf{sk} = x$.
- $\mathsf{HintGen}(\mathsf{CRS}, \mathsf{sk}_i, N, i)$: *Output*

$$\begin{aligned}
\mathsf{hint}_i = \Big(& [\mathsf{sk}_i L_i(\tau)]_1, [\mathsf{sk}_i (L_i(\tau) - L_i(0))]_1, \\
& \Big[\mathsf{sk}_i \frac{L_i^2(\tau) - L_i(\tau)}{Z(\tau)}\Big]_1, \Big[\mathsf{sk}_i \frac{L_i(\tau) - L_i(0)}{\tau}\Big]_1, \\
& \Big\{\Big[\mathsf{sk}_i \frac{L_i(\tau) L_j(\tau)}{Z(\tau)}\Big]_1\Big\}_{j \in [N]', j \neq i}\Big).
\end{aligned}$$

- $\mathsf{Preprocess}(\mathsf{CRS}, \{\mathsf{hint}_i, \mathsf{pk}_i\}_{i\in[N]})$: *Verify each hint and let* $V \subseteq [N]$ *be the set of indices with valid hints. Set* $\mathsf{sk}_0 = 1, \mathsf{sk}_i = 0$ *for* $i \notin V'$. *Output*

$$\begin{aligned}
\mathsf{ak} = \Big(& V, \{\mathsf{pk}_i\}_{i\in V'}, \{\mathsf{hint}_i^{(2)}\}_{i\in V'}, \{\mathsf{hint}_i^{(3)}\}_{i\in V'}, \\
& \{\mathsf{hint}_i^{(4)}\}_{i\in V'}, \Big\{\sum_{j\in V', j\neq i} \mathsf{hint}_{j,i}^{(5)}\Big\}_{i\in V'}\Big), \\
\mathsf{vk} = \Big(& \sum_{i\in V'} \mathsf{hint}_i^{(1)}, [Z(\tau)]_2\Big) = (C, Z).
\end{aligned}$$

- $\mathsf{EpochRepr}(e)$: *Obtain committee* $\mathsf{Com}_e \subseteq [N]$ *of epoch* e, *compute* $B_{\mathsf{Com}_e}(X)$ *by interpolating* $\{(1,1), (\omega^i, 0)_{i\notin\mathsf{Com}'_e}\}$:

$$B_{\mathsf{Com}_e}(X) = \prod_{i\notin\mathsf{Com}'_e} \frac{X - \omega^i}{1 - \omega^i},$$

  and output $B_e = [B_{\mathsf{Com}_e}(\tau)]_2$.
- $\mathsf{Sign}(\mathsf{sk}_i, m, e)$: *Let* $\mathsf{tag} = \mathcal{H}(m, e)$ *and output* $\sigma_i = \mathsf{sk}_i \cdot \mathsf{tag}$.
- $\mathsf{PartVerify}(\mathsf{pk}_i, \sigma_i, m, e)$: *Let* $\mathsf{tag} = \mathcal{H}(m, e)$ *and output 1 iff* $\mathsf{pk}_i \circ \mathsf{tag} = [1]_1 \circ \sigma_i$.
- $\mathsf{Aggr}(\mathsf{CRS}, \mathsf{ak}, m, \{\sigma_i\}_{i\in S}, t, e)$: *Discard signatures from those whose hints did not verify (under* HintVerify*), whose signatures did not verify (under* PartVerify*), and who do not belong to* Com_e. *Let* S *be the set of remaining signatures, and proceed as follows:*
  1. *Compute* $B(X)$ *by interpolating* $\{(1,1), (\omega^i, 0)_{i\notin S'}\}$:

$$B(X) = \prod_{i\notin S'} \frac{X - \omega^i}{1 - \omega^i},$$

     and compute $B = [\sum_{i=0}^N B(\omega^i) L_i(\tau)]_2$.
  2. *Compute* $\mathsf{aPK} = \frac{1}{N+1}(\sum_{i\in S} B(\omega^i)\mathsf{pk}_i + [1]_1)$.

3. *Compute*

$$Q_Z = [Q_Z(\tau)]_1 = \sum_{i=0}^{N} B(\omega^i) \cdot (\mathsf{ak}_i^{(4)} + \mathsf{ak}_i^{(6)}),$$

$$Q_x = [Q_x(\tau)]_1 = \sum_{i=0}^{N} B(\omega^i) \cdot \mathsf{ak}_i^{(5)},$$

$$\hat{Q}_x = [Q_x(\tau) \cdot \tau]_1 = \sum_{i=0}^{N} B(\omega^i) \cdot \mathsf{ak}_i^{(3)}.$$

4. *Compute* $\sigma^* = \frac{1}{N+1}(\sum_{i\in S} B(\omega^i)\sigma_i + \mathsf{tag})$.
5. *Compute* $\hat{B}(X) = X^{t+1}B(X)$ *and set* $\hat{B} = [\hat{B}(\tau)]_1$.
6. *Compute* $\bar{B}(X) = \prod_{i\in \mathsf{Com}_e \setminus S} \frac{X-\omega^i}{1-\omega^i}$ *and set* $\bar{B} = [\bar{B}(\tau)]_1$.
7. *Compute* $Q_0(X) = \frac{B(X)-1}{X-1}$ *and set* $Q_0 = [Q_0(\tau)]_1$.
8. *Output* $\sigma = (B, -\mathsf{aPK}, -Q_Z, -Q_x, \hat{Q}_x, \sigma^*, -\hat{B}, -\bar{B}, -Q_0)^\top$.

- $\mathsf{Verify}(\mathsf{vk}, m, \sigma, t, e)$: *Let* $\mathsf{tag} = \mathcal{H}(m, e)$ *and set* $Z_0 = [\tau - 1]_2$, *compute*

$$\boldsymbol{A} = \begin{pmatrix} C & [1]_2 & Z & [\tau]_2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & [\tau]_2 & [1]_2 & 0 & 0 & 0 & 0 \\ 0 & \mathsf{tag} & 0 & 0 & 0 & [1]_1 & 0 & 0 & 0 \\ [\tau^{t+1}]_1 & 0 & 0 & 0 & 0 & 0 & [1]_2 & 0 & 0 \\ [1]_1 & 0 & 0 & 0 & 0 & 0 & 0 & B_e & 0 \\ [1]_1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & Z_0 \end{pmatrix}$$

$$\boldsymbol{b} = ([0]_T, [0]_T, [0]_T, [0]_T, [0]_T, [1]_T)^\top$$

and output 1 iff $\boldsymbol{A} \circ \sigma = \boldsymbol{b}$. *This corresponds to Equation 2 since it is fully linear, i.e., each equation only consists of pairing computation, while each pairing has at least one public input (we use colors to demonstrate this).*

To verify the well-formedness of each hint, we define the HintVerify algorithm as follows.

$\mathsf{HintVerify}(\mathsf{CRS}, \mathsf{hint}_i, \mathsf{pk}_i) \to 1/0$: Parse $\mathsf{hint}_i = (h_1, h_2, h_3, h_4, \{h_5^{(j)}\}_{j\in[N]', j\neq i})$. Output 1 iff

$$h_1 \circ [1]_2 = \mathsf{pk}_i \circ [L_i(\tau)]_2$$
$$h_2 \circ [1]_2 = \mathsf{pk}_i \circ [L_i(\tau) - L_i(0)]_2$$
$$h_3 \circ [1]_2 = \mathsf{pk}_i \circ \left[\frac{L_i^2(\tau) - L_i(\tau)}{Z(\tau)}\right]_2$$
$$h_4 \circ [1]_2 = \mathsf{pk}_i \circ \left[\frac{L_i(\tau) - L_i(0)}{\tau}\right]_2$$
$$h_{5,j} \circ [1]_2 = \mathsf{pk}_i \circ \left[\frac{L_i(\tau)L_j(\tau)}{Z(\tau)}\right]_2, \forall j \in [N]' \wedge j \neq i$$

Remark 1. *We assume that the* Setup *algorithm also outputs the commitments of the Lagrange basis polynomials* $[L_i(\tau)]_{1,2}$. *This is because almost all steps of our scheme use these values, and preprocessing them can significantly reduce the related computational cost. Since these values can be easily computed and publicly verified by* CRS, *this assumption will not affect the security of our scheme.*

5 MAKING C-STS PRACTICAL

In this section, we analyze the efficiency of Construction 1. We first review the time complexities of several fundamental polynomial operations:

- Given a polynomial $f(X)$ with $f(0) \neq 0$, its multiplicative inverse modulo X^n can be computed in $O(n \log n)$ time by Newton's iteration and the Fast Fourier Transform (FFT).
- For a polynomial $f(X)$ of degree n, evaluating $f(X)$ at n arbitrary distinct points requires $O(n \log^2 n)$ time by a divide-and-conquer approach. Alternatively, if the evaluation points are the N-th roots of unity ($N \geq n$), the evaluation can be computed in $O(N \log N)$ time by FFT.
- Let ω be an N-th root of unity and $S \subseteq [N]$ with $|S| = n$. Computing the coefficients of $f_S(X) = \prod_{i\in S}(X - \omega^i)$ by divide-and-conquer requires $O(n \log^2 n)$ time. However, $f_S(X)$ can also be obtained by computing the inverse of $-\frac{Z(X)}{f_S(X)}$ modulo X^{n+1}, which takes $O((N-n)\log^2(N-n) + n \log n)$ time and is asymptotically faster when $N - n \ll n$.

Let $n = |\mathsf{Com}_e|$ be the size of the epoch e's committee and $s = |S|$ be the number of valid partial signatures; then for any valid S we have $t \leq s \leq n$. Under the assumption that $N \gg n$, the efficiency of Construction 1 can be analyzed as follows:

- The complexity of EpochRepr is $O(N \log N)$, since obtaining $[B_{\mathsf{Com}_e}(\tau)]_2$ requires to compute $B_{\mathsf{Com}_e}(X) = \prod_{i\in[N]\setminus\mathsf{Com}_e} \frac{X-\omega^i}{1-\omega^i}$, which has complexity $O(N \log N)$ when $N \gg n$.
- The complexity of Aggr is also $O(N \log N)$:
  1. Computing $B(X)$ has complexity $O(N \log N)$ (similar to computing $B_{\mathsf{Com}_e}(X)$).
  2. Evaluating $B(\omega^i)_{i\in S}$ from $B(X)$ has complexity $O(N \log N)$ since $\deg(B) = N - s$.
  3. Computing $[B(\tau)]_2 = [\sum_{i=0}^{N} B(\omega^i)L_i(\tau)]_2$ is $O(n)$ since there are only $O(n)$ non-zero $B(\omega^i)$. Similarly, the complexity of computing $aPK, \sigma^*, [Q_Z(\tau)]_1, [Q_x(\tau)]_1, [\hat{Q}_x(\tau)]_1$ is also $O(n)$.
  4. Since $\hat{B}(X) = X^{t+1}B(X)$, $[\hat{B}(\tau)]_1$ can be computed directly from the coefficients of $B(X)$, which has complexity $O(N)$.
  5. Computing $\bar{B}(X)$ has complexity $O(n \log^2 n)$, and computing $[\bar{B}(\tau)]_1$ has complexity $O(n)$.
  6. Computing the coefficients of $Q_0(X)$ and $[Q_0(\tau)]_1$ has complexity $O(N)$.

The analysis above demonstrates that although we have successfully removed the Plonk-style proofs in Dyna-hinTS, the efficiency of Construction 1 cannot meet our requirements in large-scale systems, where the global system size is much larger than that of each epoch's committee. Therefore, we propose improvements to Construction 1 to make it practical even if N is very large. Specifically, we reduce the complexity of EpochRepr and Aggr from $O(N \log N)$ to $O(n \log^2 n)$, which significantly improves their efficiency when $N \gg n$. We also reduce the complexity of HintGen from $O(N^2)$ to $O(N)$, which is of independent interest. We describe the details of all optimizations below.

5.1 Improve Epoch Commitment

Recall the equation of the subset check

$$[1]_1 \circ \boxed{[B(\tau)]_2} = [B_{\mathsf{Com}_e}(\tau)]_2 \circ \boxed{[\bar{B}(\tau)]_1} \tag{3}$$

Since $B_{\mathsf{Com}_e}(X)$ has $N-n$ zeros, we must compute it in $O(N \log N)$. In contrast, its "complementary" polynomial $C_{\mathsf{Com}_e}(X)$ with respect

to the vanishing polynomial $Z(X)$, i.e.,

$$C_{\mathsf{Com}_e}(X) = \prod_{i \in \mathsf{Com}'_e} (X - \omega^i),$$

has only $n+1$ zeros and can be computed in $O(n \log^2 n)$. Thus, multiplying $C_{\mathsf{Com}_e}(\tau)$ on both sides of equation 3 we have

$$[C_{\mathsf{Com}_e}(\tau)]_1 \circ [B(\tau)]_2 = [k \cdot Z(\tau)]_2 \circ [\bar{B}(\tau)]_1 \tag{4}$$

where k is some constant value (we compute it below) and can be moved into the computation of $[\bar{B}(\tau)]_1$. Notice that equation 4 is still linear because the commitment of the vanishing polynomial $[Z(\tau)]_2$ is public. This allows us to replace the output of $\mathsf{EpochRepr}(e)$ by $[C_{\mathsf{Com}_e}(\tau)]_1$ without compromising the security properties of the scheme. The modified $\bar{B}(X)$ can be computed as follows and remains $O(n \log^2 n)$:

$$\bar{B}(X) = \frac{C_{\mathsf{Com}_e}(X) \cdot B(X)}{Z(X)} = \frac{\prod_{i \in \mathsf{Com}'_e}(X - \omega^i) \cdot \prod_{i \notin S, i \neq 0} \frac{X - \omega^i}{1 - \omega^i}}{\prod_i (X - \omega^i)}$$
$$= \frac{\prod_{i \in \mathsf{Com}_e \setminus S}(X - \omega^i)}{\prod_{i \notin S, i \neq 0}(1 - \omega^i)} = \prod_{i \in \mathsf{Com}_e \setminus S} (X - \omega^i) \cdot \frac{\prod_{i \in S}(1 - \omega^i)}{N+1}$$

5.2 Improve Signature Aggregation

There are three steps with complexity $\Omega(N)$ in the algorithm Aggr of Section 4, namely computing B, $\hat{B}$, and Q_0. We optimize these steps separately as follows.

Computing B. Since the purpose of computing $B(X)$ is to obtain $\{B(\omega^i)\}_{i \in S}$, which has only $O(n)$ non-zero elements, it is possible to reduce the complexity to $O(n)$ if we can bypass interpolating $B(X)$, i.e., to compute $\{B(\omega^i)\}_{i \in S}$ without computing $B(X)$ itself.

Let $f_k(X) = \prod_{i \neq k}(X - \omega^i) = \frac{X^{N+1} - 1}{X - \omega^k}$, then by $L_k(\omega^k) = 1$ we have $f_k(\omega^k) = \frac{N+1}{\omega^k}$. Since $B(X) = \prod_{i \notin S, i \neq 0} \frac{X - \omega^i}{1 - \omega^i}$, for each $\{\omega^k\}_{k \in S}$ we want to compute

$$B(\omega^k) = \prod_{i \notin S, i \neq 0} \frac{\omega^k - \omega^i}{1 - \omega^i} = \frac{\prod_{i \neq 0, i \neq k} \frac{\omega^k - \omega^i}{1 - \omega^i}}{\prod_{i \in S, i \neq k} \frac{\omega^k - \omega^i}{1 - \omega^i}}.$$

For the numerator, we have

$$\prod_{i \neq 0, i \neq k} \frac{\omega^k - \omega^i}{1 - \omega^i} = \frac{\prod_{i \neq 0, i \neq k}(\omega^k - \omega^i)}{\prod_{i \neq 0, i \neq k}(1 - \omega^i)} = \frac{f_k(\omega^k)/(\omega^k - 1)}{f_0(1)/(1 - \omega^k)}$$
$$= \frac{N+1}{\omega^k(\omega^k - 1)} \cdot \frac{1 - \omega^k}{N+1} = -\omega^{-k}.$$

For the denominator, let $g(X) = \prod_{i \in S}(X - \omega^i)$, then $g'(\omega^k) = \prod_{i \in S, i \neq k}(\omega^k - \omega^i)$ and

$$\prod_{i \in S, i \neq k} \frac{\omega^k - \omega^i}{1 - \omega^i} = \frac{\prod_{i \in S, i \neq k}(\omega^k - \omega^i)}{\prod_{i \in S, i \neq k}(1 - \omega^i)} = (1 - \omega^k) \cdot \frac{g'(\omega^k)}{g(1)}.$$

In conclusion, we can compute $B(\omega^k)$ by

$$B(\omega^k) = \frac{g(1)}{\omega^k \cdot (\omega^k - 1) \cdot g'(\omega^k)}.$$

Since the complexity of computing $g(1)$ is $O(n)$, the complexity of computing and evaluating $g'(X)$ on S is $O(n \log^2 n)$, all $B(\omega^k)$ can be computed with complexity $O(n \log^2 n)$. Finally, using $\{B(\omega^k)\}_{k \in S}$ to compute $B = [\sum_{i \in S'} B(\omega^i) L_i(\tau)]_2$ has complexity $O(n)$.

Computing $\hat{B}$. $\hat{B}$ can be computed in a similar way to computing B since we already know that $\hat{B}(1) = 1, \hat{B}(\omega^i)_{i \notin S, i \neq 0} = 0$ and $\hat{B}(\omega^i)_{i \in S} = \omega^{i(t+1)} B(\omega^i)$. Since $\deg(\hat{B}) = (N - s) + (t + 1) \leq N$ when $s > t$, computing $\hat{B} = [\hat{B}(\tau)]_1 = [\sum_{i \in S'} \hat{B}(\omega^i) L_i(\tau)]_1$ has complexity $O(n)$.

The only exception is $\deg(\hat{B}) = N + 1$ when $s = t$, where $\hat{B}(X)$ cannot be directly represented by $L_i(X)$. Let $\hat{B}'(X) = \sum_{i \in S'} \hat{B}(\omega^i) L_i(X)$; then $\hat{B}(X) = \hat{B}'(X)$ for all ω^i, which implies $Z(X) | \hat{B}(X) - \hat{B}'(X)$. Since $\deg(\hat{B}') \leq N$ and $\deg(Z) = N + 1$, it must be

$$\hat{B}(X) = \sum_{i \in S'} \hat{B}(\omega^i) L_i(X) + k \cdot Z(X),$$

where k equals to the leading coefficient of $B(X)$ and can be computed as

$$k = \prod_{i \notin S, i \neq 0} \frac{1}{1 - \omega^i} = \frac{g(1)}{N+1}.$$

Therefore, computing

$$\hat{B} = [\hat{B}(\tau)]_1 = \Big[\sum_{i \in S'} \hat{B}(\omega^i) L_i(\tau) \Big]_1 + \frac{g(1)}{N+1} \cdot [Z(\tau)]_1$$

also has complexity $O(n)$.

Computing Q_0. The method above cannot be used again for computing $Q_0 = [Q_0(\tau)]_1$ since $\deg(Q_0) = N - s - 1$ and $Q_0(X) = \frac{B(X) - 1}{X - 1}$ has no zeros on $\mathbb{H}$, which implies that the complexity of computing Q_0 is $\Omega(N)$ either by computing coefficients of $Q_0(X)$ or by $[Q_0(\tau)]_1 = [\sum_{i=0}^{N} Q_0(\omega^i) L_i(\tau)]_1$.

To address this challenge, we should review the role of Q_0: it is the KZG opening proof of $B(\omega^0) = 1$, which ensures that the dummy party $\mathsf{P}_0 \in B$. Our approach here is to **regard the 1 on the right-hand side as the evaluation of some polynomial $h(X)$ at ω^0**, i.e., we prove $B(\omega^0) = h(\omega^0)$ for a polynomial $h(X)$ with $h(\omega^0) = 1$ publicly verifiable. This clearly does not affect the statement we need to prove, but by carefully choosing $h(X)$, the computation of $Q_0 = [Q_0(\tau)]_1$ may be significantly simplified. Since the vast majority of $\mathbb{H}$ are zeros of $B(X)$, we would like $h(X)$ to have the same property, and $h(X) = L_0(X)$ becomes the most natural choice. By defining

$$Q_0(X) = \frac{B(X) - L_0(X)}{X - 1}, \tag{5}$$

we have $Q_0(\omega^i) = 0$ for $i \notin S, i \neq 0$, i.e., there are only $O(n)$ non-zero $Q_0(\omega^i)$.

For $i \in S$, we know that $Q_0(\omega^i) = \frac{B(\omega^i)}{\omega^i - 1}$ which can be computed in $O(n)$, and for $Q_0(1)$ we have $Q_0(1) = B'(1) - L'_0(1)$. To compute $B'(1)$ and $L'_0(1)$, notice that for any $p(X) = \prod_i p_i(X)$ we have

$$p'(X) = p(X) \cdot \sum_i \frac{p'_i(X)}{p_i(X)}.$$

Since $B(X) = \prod_{i \notin S, i \neq 0} p_i(X)$ and $L_0(X) = \prod_{i \neq 0} p_i(X)$ where $p_i(X) = \frac{X - \omega^i}{1 - \omega^i}$ satisfies $p_i(1) = 1$ and $p'_i(1) = \frac{1}{1 - \omega^i}$, we have

$$B'(1) = B(1) \cdot \sum_{i \notin S, i \neq 0} \frac{p'(1)}{p(1)} = \sum_{i \notin S, i \neq 0} \frac{1}{1 - \omega^i}$$

$$L'_0(1) = L_0(1) \cdot \sum_{i \neq 0} \frac{p'(1)}{p(1)} = \sum_{i \neq 0} \frac{1}{1 - \omega^i}$$

which concludes that

$$Q_0(1) = \sum_{i \in S} \frac{1}{\omega^i - 1}$$

and can also be computed in $O(n)$. The result above allows us to compute $Q_0 = [\sum_{i \in S'} Q_0(\omega^i) L_i(\tau)]_1$ in $O(n)$ since there are only $O(n)$ non-zero $Q_0(\omega^i)$.

Finally, the verification equation corresponding to equation 5 is

$$[1]_1 \circ \boxed{[B(\tau)]_2} = [\tau - 1]_2 \circ \boxed{[Q_0(\tau)]_1} + [L_0(\tau)]_1 \circ [1]_2,$$

which remains linear.

By applying the optimizations described above, our full signature verification equations can be summarized as

$$\begin{aligned}
[SK(\tau)]_1 \circ \boxed{[B(\tau)]_2} &= [1]_2 \circ \boxed{\mathsf{aPK}} + [Z(\tau)]_2 \circ \boxed{[Q_Z(\tau)]_1} \\
&\quad + [\tau]_2 \circ \boxed{[Q_x(\tau)]_1} \\
[\tau]_2 \circ \boxed{[Q_x(\tau)]_1} &= [1]_2 \circ \boxed{[\hat{Q}_x(\tau)]_1} \\
\mathsf{tag} \circ \boxed{\mathsf{aPK}} &= [1]_1 \circ \boxed{\sigma^*} \\
[\tau^{t+1}]_1 \circ \boxed{[B(\tau)]_2} &= [1]_2 \circ \boxed{[\hat{B}(\tau)]_1} \\
[C_{\mathsf{Com}_e}(\tau)]_1 \circ \boxed{[B(\tau)]_2} &= [Z(\tau)]_2 \circ \boxed{[\bar{B}(\tau)]_1} \\
[1]_1 \circ \boxed{[B(\tau)]_2} &= [\tau - 1]_2 \circ \boxed{[Q_0(\tau)]_1} + [L_0(\tau)]_1 \circ [1]_2
\end{aligned} \tag{6}$$

which has the complexity $O(n \log^2 n)$.

5.3 Improve Silent Setup

Although this work shares a common silent setup phase with [26, 27, 29], we notice that the optimizations to this phase are rarely mentioned. This is likely because the setup is one-time and trivially parallelizable; hence, its cost does not critically affect the usability of the underlying schemes.

Nevertheless, we provide some improvements to the silent setup phase in this section, which do not relate to specific threshold signatures or encryption schemes. Therefore, they can also be applied to [26, 27, 29] and may be of independent interest.

The silent setup phase in these schemes consists of the Setup, KeyGen, HintGen, Preprocess algorithms, among which we focus on the optimization of HintGen since the other algorithms are hard to optimize at the mathematical level. To generate hint_i, one must compute $H_i = (h_1, h_2, h_3, h_4, \{h_{5,j}\}_{j \in [N]', j \neq i})$ in advance, where

$$\begin{aligned}
h_1 &= [L_i(\tau)]_1, h_2 = [L_i(\tau) - L_i(0)]_1, \\
h_3 &= \left[\frac{L_i^2(\tau) - L_i(\tau)}{Z(\tau)}\right]_1, h_4 = \left[\frac{L_i(\tau) - L_i(0)}{\tau}\right]_1, \\
h_{5,j} &= \left[\frac{L_i(\tau) L_j(\tau)}{Z(\tau)}\right]_1, \forall j \in [N]' \wedge j \neq i.
\end{aligned}$$

In practice, each party may compute its own H_i by itself, or the aggregator may compute all of them and deliver each H_i to the i-th party. In any case, the i-th party multiplies its own sk_i with all the values in H_i and sends them to the aggregator. Therefore, we only focus on how to compute H_i efficiently below.

We denote the multi-scalar multiplication of k $\mathbb{G}_i$ ($i = 1, 2$) points by $\mathrm{msm}_i(k)$. The naive approach computes each polynomial in H_i and its commitment:

- $h_1 = [L_i(\tau)]_1$ is already available. Since $[L_i(0)]_1 = [\frac{1}{N+1}]_1$ is a constant for all $i \in [N]'$, we also consider it publicly available, hence computing h_2 requires 1 $\mathbb{G}_1$ addition.
- Computing h_3 and h_4 requires to obtain the coefficients of $h_3(X)$ and $h_4(X)$ first, and $\mathrm{msm}_1(N)$ each for their commitments since they have degree $N - 1$.
- Similarly, computing $h_{5,j}$ requires to obtain the coefficients of $h_{5,j}(X)$ first, and then $\mathrm{msm}_1(N)$. These steps apply to all $j \in [N]', j \neq i$.

In conclusion, the total complexity of computing H_i is $O(N^2)$, and can be reduced to $O(N^2/\log N)$ when applying a faster MSM algorithm (e.g., Pippenger's method [35]).

Our observation is that this complexity can be further reduced to $O(N)$. Specifically, we compute h_3, h_4, h_5 as follows:

- By $\sum_{j \in [N]'} L_j(X) = 1$, we have

$$\sum_{j \in [N]', j \neq i} \frac{L_i(X) L_j(X)}{Z(X)} + \frac{L_i^2(X)}{Z(X)} = \frac{L_i(X)}{Z(X)},$$

  namely $h_3(X) = -\sum h_{5,j}(X)$. Therefore, we can compute $h_3 = -\sum h_{5,j}$ by $(N-1)\mathbb{G}_1$ additions after we obtain h_5.
- For $h_4(X)$, we have

$$\begin{aligned}
\frac{L_i(X) - L_i(0)}{X} &= \frac{1}{X}\left(\frac{\omega^i}{N+1} \cdot \frac{X^{N+1} - 1}{X - \omega^i} - \frac{1}{N+1}\right) \\
&= \frac{1}{X} \cdot \frac{\omega^i (X^{N+1} - 1) - (X - \omega^i)}{(N+1)(X - \omega^i)} \\
&= \frac{\omega^i X^N - 1}{(N+1)(X - \omega^i)} \\
&= \frac{(X^{N+1} - 1) - (X^{N+1} - \omega^i X^N)}{(N+1)(X - \omega^i)} \\
&= \frac{L_i(X)}{\omega^i} - \frac{X^N}{N+1}
\end{aligned}$$

  Therefore, $h_4 = \frac{[L_i(\tau)]_1}{\omega^i} - \frac{[\tau^N]_1}{N+1}$ and can be computed by 2 $\mathbb{G}_1$ multiplications and 1 $\mathbb{G}_1$ addition.
- For $h_{5,j}(X)$, since

$$\frac{L_i(X) L_j(X)}{Z(X)} = \frac{\omega^i \omega^j}{(N+1)^2} \cdot \frac{X^{N+1} - 1}{(X - \omega^i)(X - \omega^j)},$$

  we have

$$\begin{aligned}
h_{5,j}(\omega^i) &= \frac{1}{N+1} \cdot \frac{\omega^j}{\omega^i - \omega^j} \\
h_{5,j}(\omega^j) &= \frac{1}{N+1} \cdot \frac{\omega^i}{\omega^j - \omega^i}
\end{aligned}$$

  and $h_{5,j}(\omega^k) = 0$ for all $k \neq i, j$. Therefore,

$$\begin{aligned}
[h_{5,j}(\tau)]_1 &= \sum_{k=0}^{N} h_{5,j}(\omega^k) [L_k(\tau)]_1 \\
&= h_{5,j}(\omega^i) [L_i(\tau)]_1 + h_{5,j}(\omega^j) [L_j(\tau)]_1.
\end{aligned}$$

  Notice that $h_{5,j}(\omega^i) + h_{5,j}(\omega^j) = -\frac{1}{N+1}$, we can further write

$$[h_{5,j}(\tau)]_1 = -\frac{[L_i(\tau)]_1}{N+1} + h_{5,j}(\omega^j)([L_j(\tau)]_1 - [L_i(\tau)]_1)$$

where the first term is constant across all $h_{5,j}$. This allows us to compute each $h_{5,j}$ by 1 $\mathbb{G}_1$ multiplication and 2 $\mathbb{G}_1$ additions, with a single multiplication for computing $-\frac{[L_i(\tau)]_1}{N+1}$.

In conclusion, the total complexity of our approach to compute H_i is $(N+3)\mathbb{G}_1$ multiplications and $(3N+1)\mathbb{G}_1$ additions, providing a significant improvement over the naive approach.

6 THE FINAL C-STS CONSTRUCTION

In this section, we propose our core contribution, i.e., the practical silent threshold signature scheme for dynamic committees. Our final construction is demonstrated in Section 6.1, and we analyze its security properties in Section 6.2.

6.1 Construction of Practical c-STS Scheme

Our final construction combines the construction in Section 4 and the improvements in Section 5. For clarity, differences from Construction 1 are shown in orange.

Construction 2 (Practical c-STS). *We specify each algorithm of practical c-STS as follows.*

- *The* Setup, KeyGen, HintGen *and* Preprocess *algorithms are the same as Construction 1.*
- EpochRepr(e): *Obtain committee* $\mathsf{Com}_e \subseteq [N]$ *of epoch* e, *compute* $C_{\mathsf{Com}_e}(X) = \prod_{i\in\mathsf{Com}'_e}(X-\omega^i)$ *and output* $B_e = [C_{\mathsf{Com}_e}(\tau)]_1$.
- Sign(sk_i, m, e): *Let* $\mathsf{tag} = \mathcal{H}(m,e)$ *and output* $\sigma_i = \mathsf{sk}_i \cdot \mathsf{tag}$.
- PartVerify($\mathsf{pk}_i, \sigma_i, m, e$): *Let* $\mathsf{tag} = \mathcal{H}(m,e)$ *and output 1 iff* $\mathsf{pk}_i \circ \mathsf{tag} = [1]_1 \circ \sigma_i$.
- Aggr(CRS, ak, m, $\{\sigma_i\}_{i\in S}, t, e$): *Discard signatures from those whose hints did not verify (under* HintVerify*), whose signatures did not verify (under* PartVerify*), and who do not belong to* Com_e. *Let* S *be the set of remaining signatures and proceed as follows (recall that* $S' = S \cup \{0\}$*):*
  1. *Define* $B(1) = 1$ *and* $B(\omega^i) = 0$ *for* $i \notin S'$. *Compute* $g(X) = \prod_{i\in S}(X-\omega^i)$ *and* $\{g'(\omega^i)\}_{i\in S}$. *For* $i \in S$, *compute*
  $$B(\omega^i) = \frac{g(1)}{\omega^i \cdot (\omega^i - 1) \cdot g'(\omega^i)}$$
  and set $B = [\sum_{i\in S'} B(\omega^i) L_i(\tau)]_2$.
  2. *Compute* $\mathsf{aPK} = \frac{1}{N+1}(\sum_{i\in S'} B(\omega^i)\mathsf{pk}_i)$.
  3. *Compute*
  $$Q_Z = [Q_Z(\tau)]_1 = \sum_{i\in S'} B(\omega^i)\cdot(\mathsf{ak}_i^{(4)} + \mathsf{ak}_i^{(6)}),$$
  $$Q_x = [Q_x(\tau)]_1 = \sum_{i\in S'} B(\omega^i)\cdot \mathsf{ak}_i^{(5)},$$
  $$\hat{Q}_x = [Q_x(\tau)\cdot\tau]_1 = \sum_{i\in S'} B(\omega^i)\cdot \mathsf{ak}_i^{(3)}.$$
  4. *Compute* $\sigma^* = \frac{1}{N+1}(\sum_{i\in S} B(\omega^i)\sigma_i + \mathsf{tag})$.
  5. *Let* $\hat{B}(1) = 1$; *for* $i \in S$, *compute* $\hat{B}(\omega^i)_{i\in S} = \omega^{i(t+1)} B(\omega^i)$ *and* $\hat{B}' = [\sum_{i\in S'} \hat{B}(\omega^i) L_i(\tau)]_1$. *Set*
  $$\hat{B} = [\hat{B}(\tau)]_1 = \begin{cases} \hat{B}', & |S| > t \\ \hat{B}' + \dfrac{g(1)}{N+1}\cdot[Z(\tau)]_1, & |S| = t \end{cases}$$
  6. *Compute* $\bar{B}(X) = \frac{g(1)}{N+1}\prod_{i\in\mathsf{Com}_e\setminus S}(X-\omega^i)$ *and set* $\bar{B} = [\bar{B}(\tau)]_1$.
  7. *Let* $Q_0(X) = \frac{B(X)-L_0(X)}{X-1}$ *(but do not compute* $Q_0(X)$ *and* $Q_0 = [Q_0(\tau)]_1$ *directly). For* $i \in S$, *compute* $Q_0(\omega^i)_{i\in S} = \frac{B(\omega^i)}{\omega^i-1}$ *and* $Q_0(1) = \sum_{i\in S}\frac{1}{\omega^i-1}$. *Finally, set* $Q_0 = [Q_0(\tau)]_1 = [\sum_{i\in S'} Q_0(\omega^i)L_i(\tau)]_1$.
  8. *Output* $\sigma = (B, -\mathsf{aPK}, -Q_Z, -Q_x, \hat{Q}_x, \sigma^*, -\hat{B}, -\bar{B}, -Q_0)^\top$.
- Verify($\mathsf{vk}, m, \sigma, t, e$): *Let* $\mathsf{tag} = \mathcal{H}(m,e)$, *set* $Z_0 = [\tau - 1]_2$ *and* $E = [L_0(\tau)]_T$, *compute*

$$\boldsymbol{A} = \begin{pmatrix} C & [1]_2 & Z & [\tau]_2 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & [\tau]_2 & [1]_2 & 0 & 0 & 0 & 0 \\ 0 & \mathsf{tag} & 0 & 0 & 0 & [1]_1 & 0 & 0 & 0 \\ [\tau^{t+1}]_1 & 0 & 0 & 0 & 0 & 0 & [1]_2 & 0 & 0 \\ B_e & 0 & 0 & 0 & 0 & 0 & 0 & Z & 0 \\ [1]_1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & Z_0 \end{pmatrix}$$

$$\boldsymbol{b} = ([0]_T, [0]_T, [0]_T, [0]_T, [0]_T, E)^\top$$

and output 1 iff $\boldsymbol{A}\circ\sigma = \boldsymbol{b}$.

6.2 Security Analysis

The following theorem guarantees the security of our c-STS scheme.

Theorem 6.1. *Construction 2 satisfies the correctness (Definition A.1) and unforgeability (Definition A.2).*

We defer the formal proof to Appendix B.

6.3 Remarks on Epoch Setup and Verification Complexity

In our scheme, achieving $O(1)$ complexity for signature verification intrinsically relies on the verifier having access to B_e. Although we assume this in Section 3, how to obtain B_e still requires careful consideration. In fact, whether the verifier recomputes B_e or verifies an additional proof provided by a third party, this step inherently requires $\Omega(n)$ due to reading Com_e (which is $O(n)$).

To resolve this without imposing a heavy burden on the verifier during per-message verification, we follow the paradigm established in [29], i.e., decoupling the computation or verification of B_e from the per-message signature verification. Specifically, trusting B_e is treated as a one-time operation per epoch rather than a per-signature overhead. This can be instantiated in two ways:

- At the beginning of a new epoch, the verifier computes B_e itself, which requires $O(n\log^2 n)$ field operations + $\mathsf{msm}_1(n)$, and caches B_e locally. For the remainder of the epoch, any signature verified against this committee takes strict $O(1)$ time. Thus, the setup cost is amortized over all signatures within the epoch.
- In on-chain environments, the validation of B_e can be handled by a smart contract. An untrusted off-chain entity computes B_e and submits it to the contract alongside a succinct proof (e.g., a KZG opening) attesting that B_e correctly corresponds to Com_e. The smart contract performs the verification once and stores B_e in its state, allowing anyone to verify signatures in epoch e in $O(1)$.

In summary, while the end-to-end process inherently involves an $O(n)$ step to link the dynamic committee to the commitment,

our design isolates this cost to the epoch boundary. Consequently, the signature verification complexity remains $O(1)$.

7 DEFINITIONS OF C-STE

As another contribution of this work, we formally define a new primitive, namely the silent threshold encryption scheme for dynamic committees (c-STE), in this section, and give a practical construction in Section 8. Our definition adheres to the structure of [26, 27, 29]. We extend silent threshold encryption to support dynamic committees, in which the authorized committee is a subset of the entire set of parties that changes dynamically over epochs, and messages encrypted for a specific epoch can be decrypted only by the corresponding committee.

Definition 7.1 (c-STE). A silent threshold encryption scheme for dynamic committees consists of a tuple of algorithms Σ = (Setup, KeyGen, HintGen, Preprocess, EpochRepr, Enc, PartDec, PartVerify, DecAggr) with the following syntax:

- $\mathsf{CRS} \leftarrow \mathsf{Setup}(1^\lambda, N)$: On input the security parameter λ and an upper bound N on the maximum number of users, the Setup algorithm outputs a common reference string CRS.
- $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$: On input the security parameter λ, the KeyGen algorithm outputs a key pair $(\mathsf{pk}, \mathsf{sk})$.
- $\mathsf{hint}_i \leftarrow \mathsf{HintGen}(\mathsf{CRS}, sk, N, i)$: On input the CRS, the secret key sk, the number of parties N, and a position $i \in [N]$, the HintGen algorithm outputs a hint hint_i.
- $(\mathsf{ak}, \mathsf{ek}) \leftarrow \mathsf{Preprocess}(\mathsf{CRS}, \{\mathsf{hint}_i, pk_i\}_{i\in[N]})$: On input the CRS, all pairs $\{\mathsf{hint}_i, pk_i\}_{i\in[N]}$, the Preprocess algorithm outputs an aggregation key ak and an encryption key ek.
- $B_e \leftarrow \mathsf{EpochRepr}(e)$: On input epoch e, the EpochRepr algorithm outputs a mathematical representation B_e of the epoch e's committee $\mathsf{Com}_e \subseteq [N]$.
- $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{ek}, \mathsf{msg}, t, e)$: On input an encryption key ek, a message msg, a threshold t, and epoch e, the Enc algorithm outputs a ciphertext ct.
- $\sigma \leftarrow \mathsf{PartDec}(\mathsf{sk}, \mathsf{ct}, e)$: On input a secret key sk, a ciphertext ct, and epoch e, the PartDec algorithm outputs a partial decryption σ.
- $1/0 \leftarrow \mathsf{PartVerify}(\mathsf{pk}, \sigma, \mathsf{ct}, e)$: On input a ciphertext ct, a partial decryption σ, a public key pk, and epoch e, the PartVerify algorithm outputs 1 iff the partial decryption verifies and corresponds to epoch e.
- $\mathsf{msg} \leftarrow \mathsf{DecAggr}(\mathsf{CRS}, \mathsf{ak}, \mathsf{ct}, \{\sigma_i\}_{i\in S}, t, e)$: On input the CRS, an aggregation key ak, a ciphertext ct, a set of partial decryptions $\{\sigma_i\}_{i\in S}$, a threshold t, and epoch e, the DecAggr algorithm outputs a message msg.

Moreover, the c-STE scheme must have the following efficiency requirements:

- The size of the encryption key ek and the ciphertext ct should be $O(1)$.
- The complexity of PartDec should be $O(1)$.
- For epoch e with the committee size $n = |\mathsf{Com}_e|$, the complexity of EpochRepr should be $O(n \cdot \mathrm{polylog}(n))$.
- Given the commitment of a committee B_e, the complexity of Enc should be $O(1)$, and the complexity of DecAggr should be $O(n \cdot \mathrm{polylog}(n))$.

We also define the correctness and semantic security of c-STE through two games between a challenger and an adversary $\mathcal{A}$. We defer the formal definition to Appendix C.

8 C-STE CONSTRUCTION

By eliminating the Plonk-style proofs, our c-STS scheme obtains a fully linear signature verification. This allows us to leverage the compiler proposed by [27] to construct a c-STE scheme from our c-STS scheme, and the c-STE scheme naturally inherits all the efficiency advantages of our c-STS scheme. We propose this construction of the practical c-STE scheme and analyze its security properties in this section.

8.1 Construction of Practical c-STE Scheme

Construction 3 (Practical c-STE). *We specify each algorithm of practical c-STE as follows, and we simplify the description of the algorithms that remain unchanged from Construction 2.*

- *The* Setup, KeyGen, HintGen, Preprocess *and* EpochRepr *algorithms are the same as Construction 2.*
- $\mathsf{Enc}(\mathsf{ek}, \mathsf{msg}, t, e)$*: Sample* $u \leftarrow_\$ \mathbb{Z}_p$ *and let* $\mathsf{tag} = \mathcal{H}(u, e)$*. Compute the matrix* $\boldsymbol{A}$ *and vector* $\boldsymbol{b}$ *as the* Verify *algorithm of Construction 2. Sample a vector* $\boldsymbol{s} \leftarrow_\$ \mathbb{Z}_p^6$ *and output* $\mathsf{ct} = (u, \boldsymbol{s}^\top \cdot \boldsymbol{A}, \boldsymbol{s}^\top \cdot \boldsymbol{b} + \mathsf{msg})$.
- $\mathsf{PartDec}(\mathsf{sk}_i, \mathsf{ct}, e)$*: Parse* $\mathsf{ct} = (ct_1, \boldsymbol{ct}_2, ct_3)$*, let* $\mathsf{tag} = \mathcal{H}(ct_1, e)$ *and output* $\sigma_i = \mathsf{sk}_i \cdot \mathsf{tag}$.
- $\mathsf{PartVerify}(\mathsf{pk}_i, \sigma_i, \mathsf{ct}, e)$*: Parse* $\mathsf{ct} = (ct_1, \boldsymbol{ct}_2, ct_3)$*, let* $\mathsf{tag} = \mathcal{H}(ct_1, e)$ *and output 1 iff* $\mathsf{pk}_i \circ \mathsf{tag} = [1]_1 \circ \sigma_i$.
- $\mathsf{DecAggr}(\mathsf{CRS}, \mathsf{ak}, \mathsf{ct}, \{\sigma_i\}_{i\in S}, t, e)$*: Discard partial decryptions from those whose hints did not verify (under* HintVerify*), whose partial decryption did not verify (under* PartVerify*), and who do not belong to* Com_e*. Let S be the set of remaining partial decryptions and proceed as follows: Parse* $\mathsf{ct} = (ct_1, \boldsymbol{ct}_2, ct_3)$*, use the* Aggr *algorithm of Construction 2 to compute* $\boldsymbol{w} = \mathsf{Aggr}(\mathsf{CRS}, \mathsf{ak}, ct_1, \{\sigma_i\}_{i\in S}, t, e)$ *and output* $\mathsf{msg}^* = ct_3 - \boldsymbol{ct}_2 \circ \boldsymbol{w}$.

8.2 Security Analysis

The following theorem guarantees the security of our c-STE scheme.

Theorem 8.1. *Construction 3 satisfies the correctness (Definition C.1) and semantic security (Definition C.2).*

We defer the formal proof to Appendix D.

9 IMPLEMENTATION AND EVALUATION

We implement our practical c-STS and c-STE schemes in Rust and evaluate their performance.

Experimental setup. We use the arkworks library [1] for elliptic curve and other mathematical operations (including the implementation of Pippenger's method [35] for the multi-scalar multiplication of curve points). We choose BLS12-381 as the underlying pairing-friendly elliptic curve, where an element in $\mathbb{Z}_p, \mathbb{G}_1, \mathbb{G}_2$ is of size

32, 48, 96 bytes, respectively. For the total number of parties N[4], we use $N \approx 2^{20}$ as a benchmark upper bound for evaluation[5]. We always set the committee threshold $t = n/2$ for simplicity.

For the sake of fairness, we only implement the single-thread version of our schemes (although there are obvious opportunities for parallelism) and run it on a machine equipped with an Intel Core Ultra 7 265K processor (3.90 GHz) and 64 GB of RAM, running Windows 11 with WSL2 and Ubuntu 20.04 LTS.

9.1 Evaluation of c-STS Scheme

We evaluate the performance of our c-STS scheme and compare it with the Rust implementation of Dyna-hinTS [2].

Epoch committee representation. The representation of epoch committees has a constant size in both Dyna-hinTS (1 $\mathbb{G}_2$, 96 bytes) and our scheme (1 $\mathbb{G}_1$, 48 bytes). As a comparison, a direct representation of a committee of size n in the BLS multi-signature scheme can use $\min(N, \lceil n \log_2 N \rceil)$ bits (2560 bytes under the scale of Ethereum).

Computing the representation of epoch committees in Dyna-hinTS only requires n $\mathbb{G}_2$ point additions. In comparison, our scheme incurs a higher cost for computing the corresponding polynomial and commitment (although they are in $\mathbb{G}_1$), which has complexity $O(n \log^2 n)$. We compare the time in Table 1 and argue that this overhead is acceptable when n is within a reasonable range. In return, this representation enables a substantial optimization of the aggregation time, which we demonstrate below.

Table 1: Time to compute the representation of epoch committees (ms)

n	Dyna-hinTS	This work	Overhead
128	0.224	3.326	14.84×
256	0.444	6.316	14.23×
512	0.926	12.012	12.97×
1024	1.842	22.698	12.32×
2048	3.720	45.840	12.32×

Partial sign and verification time. Computing and verifying a partial signature takes <0.2ms and 1.5ms respectively, since it is identical to computing and verifying a BLS signature.

Signature aggregation time. According to the analysis in 5, the cost of signature aggregation can be summarized as follows:

- Computing $g(X), g(\omega^i)$ and $B(\omega^i)$ require $O(n \log^2 n)$ field operations, and computing B requires $\mathrm{msm}_2(n)$.
- Computing aPK requires $\mathrm{msm}_1(n)$.
- Computing $Q_Z, Q_x, \hat{Q}_x$ requires $\mathrm{msm}_1(n)$ each.
- Computing σ^* requires $\mathrm{msm}_2(n)$.
- Computing $\hat{B}(\omega^i)$ requires $O(n)$ field operations, and computing $\hat{B}$ requires $\mathrm{msm}_1(n)$.
- Computing $\bar{B}(X)$ requires $O(n \log^2 n)$ field operations, and computing $\bar{B}$ requires $\mathrm{msm}_1(n)$.
- Computing $Q_0(\omega^i)$ requires $O(n)$ field operations, and computing Q_0 requires $\mathrm{msm}_1(n)$.

We conclude that our total aggregation cost is $O(n \log^2 n)$ field operations + 7 $\mathrm{msm}_1(n)$ + 2 $\mathrm{msm}_2(n)$.

We compare the time of signature aggregation of two schemes in Table 2. The results show that the aggregation time of Dyna-hinTS increases linearly with N, which is consistent with our analysis in Section 4. In contrast, the aggregation time only scales quasilinearly with n in our scheme and is not affected by N. This advantage becomes particularly significant in large-scale systems: when N = 1048576 and n = 1024, our scheme only requires 0.16 seconds to compute the aggregated signature, achieving more than 1900× improvement over Dyna-hinTS. In small-scale systems ($N \leq 4096$), our scheme also brings approximately a 11× to 70× improvement. Overall, the results demonstrate that our scheme provides high efficiency and scalability across different system sizes, making it well-suited for latency-sensitive aggregation scenarios.

Table 2: Time to compute the aggregate signature (s)

N	n	Dyna-hinTS	This work	Improvement
256	64	0.223	0.018	12.39×
1024	64	0.438	0.019	23.05×
	256	0.588	0.051	11.53×
4096	64	1.332	0.019	70.11×
	256	1.474	0.052	28.35×
	1024	2.051	0.153	13.41×
16384	64	4.799	0.019	252.6×
	256	4.938	0.051	96.82×
	1024	5.512	0.153	36.03×
65536	64	17.219	0.018	956.6×
	256	17.355	0.051	340.3×
	1024	17.896	0.154	116.2×
1048576	64	289.1	0.019	15216×
	256	289.2	0.052	5562×
	1024	289.8	0.153	1907×
	4096	291.1	0.593	490.9×

Signature size. We compare the size of the aggregated signature, the verification key, and the number of pairing operations in the verification algorithm of three schemes in Table 3. Although the BLS multi-signature scheme requires the fewest pairings, the aggregated signature must represent the set of signers (denoted by *Repr*), so the signature size scales with n when the epoch committee is given, and scales with N otherwise. Moreover, verifying a BLS multi-signature requires storing all public keys, which is a significant storage overhead that grows with N. In comparison, both Dyna-hinTS and our scheme achieve constant signature size, verification key size, and number of pairing operations. By removing Plonk-style proofs in Dyna-hinTS, our scheme reduces the signature size to 528 bytes (compared to 896 bytes in Dyna-hinTS). On the other hand, our scheme adds four pairings in verification because we check the corresponding relations by three additional linear pairing

[4]For ease of reading, in this section we include the dummy party in N so that N is a power of 2.

[5]This is close to the total number of validators in Ethereum; a system with larger N is hard to find in practice.

equations. Nevertheless, the verification time is only about 10ms (with access to the epoch committee representation).

Table 3: Complexity of signature verification

Scheme	Signature size	Verification key size	Pairings	Time
BLS	1 $\mathbb{G}_2$, *Repr*	N $\mathbb{G}_1$	2	~1.5ms
Dyna-hinTS	8 $\mathbb{G}_1$, 2 $\mathbb{G}_2$, 10 $\mathbb{Z}_p$	1 $\mathbb{G}_1$, 1 $\mathbb{G}_2$	9	7ms
This work	7 $\mathbb{G}_1$, 2 $\mathbb{G}_2$	1 $\mathbb{G}_1$, 1 $\mathbb{G}_2$	13	10ms

9.2 Evaluation of c-STE Scheme

While our c-STE construction closely parallels the c-STS scheme (and thus shares similar performance in committee representation and polynomial aggregation), we provide its evaluation to demonstrate its practical efficiency for dynamic committees.

Table 4 gives the computational time for the c-STE operations. The time to generate a ciphertext (with access to the epoch committee representation) and to compute/verify a partial decryption remains constant. The decryption time equals the signature aggregation time in the c-STS scheme (Table 2) plus 9 additional pairings (which take about 7ms) to compute $\boldsymbol{ct}_2 \circ \boldsymbol{w}$. Table 5 gives the sizes of the corresponding components, showing that the communication overhead of our c-STE is succinct.

Table 4: Time cost of c-STE scheme

Operation	Time	Remarks
Encryption	<7ms	Constant time
Partial decryption	<0.2ms	Constant time (= BLS)
Partial verification	1.5ms	Constant time (= BLS)
Decryption	Table 2 + 7ms	Aggregation + 9 pairings

Table 5: Sizes of c-STE components

Component	Size
Encryption Key	1 $\mathbb{G}_1$, 1 $\mathbb{G}_2$
Plaintext	1 $\mathbb{G}_T$
Ciphertext	1 $\mathbb{Z}_p$, 2 $\mathbb{G}_1$, 7 $\mathbb{G}_2$, 1 $\mathbb{G}_T$
Partial decryption	1 $\mathbb{G}_2$

9.3 Evaluation of Silent Setup

We also evaluate the one-time setup cost of our scheme.

Hint generation. We evaluate the impact of the optimization proposed in Section 5.3 on the time required for a single party to generate its hint during the setup phase. As analyzed in Section 5.3, our optimization reduces this computational complexity from about $O(N^2/\log N)$ to $O(N)$, which is consistent with the results in Table 6. The execution time of the naive setup becomes experimentally infeasible for $N > 4096$, and we estimate that a single party would require about 1.35×10^7 seconds (156.25 days) to compute its hint when $N = 1048576$ based on existing data. In contrast, our optimized algorithm only requires about 3 minutes to generate the same hint, demonstrating that our optimization is necessary and practically efficient.

Table 6: Time to compute a hint for a single party (s)

N	Naive approach	Our approach	Improvement
64	0.153	0.012	12.75×
256	1.830	0.049	37.35×
1024	25.72	0.191	134.66×
4096	343.1	0.772	444.4×
16384	-	3.110	-
65536	-	12.38	-
1048576	-	197.2	-

System preprocessing phase. While the Preprocess algorithm will be executed only once, it is crucial to ensure its feasibility when the system scale is large. It consists of two parts (assuming all hints are valid, namely $|V| = N$):

- Hint verification: Executing HintVerify naively needs $2N+8$ pairings per hint, which can be reduced to $\mathrm{msm}_1(N)$ + $\mathrm{msm}_2(N)$ + 2 pairings by performing random linear combinations to each side [29]. Since HintVerify must be executed N times, we introduce a further optimization that applies a second linear combination across all N verifications since their left-hand sides share the same $[1]_2$. This reduces the total number of pairings from $2N$ to $N + 1$, and the total computational overhead for this phase is reduced to $N + 1$ pairings + $N \times (\mathrm{msm}_1(N) + \mathrm{msm}_2(N))$.
- Computing ak and vk: Requiring $N^2 - 1$ and N $\mathbb{G}_1$ additions, respectively.

Therefore, the average computational overhead per hint is closely approximated by 1 pairing + N $\mathbb{G}_1$ additions + $\mathrm{msm}_1(N)$+$\mathrm{msm}_2(N)$, and we denote its time by T.

As the system scales, evaluating this setup sequentially on a single machine becomes impractical. However, this setup is fully parallelizable in practice because these computations are highly repetitive and independent. We consider the parallelism level $P = P_1P_2$ from two dimensions:

- Node Parallelism (P_1): The workload can be evenly distributed among multiple participating parties, which is highly feasible when $N \gg P_1$. The system could even assign multiple groups to execute the Preprocess algorithm redundantly to enhance trust in the results.
- Hardware Parallelism (P_2): Individual parties can leverage multi-threading to process their assigned batches. The workload distribution can also be adjusted dynamically based on heterogeneous hardware capabilities.

Therefore, the execution time of Preprocess can be reasonably estimated by $C = T \times N/P$. Table 7 shows this theoretical estimation across different system scales assuming an accessible distributed configuration of $P_1 = 128$ and $P_2 = 4$ (i.e., total parallelism $P = 512$). Under this setting, the overall execution time for an extraordinarily large scale of $N = 2^{20}$ drops to approximately 18.2 hours. Since this

is a strict one-time setup, we consider this computational overhead acceptable for large-scale distributed systems.

Table 7: Estimated setup time at $P = 512$

N	T (ms)	C (s)
1024	72.03	0.14
4096	234.07	1.87
16384	779.54	24.95
65536	2641.40	338.10
262144	9102.50	4660.48
1048576	32064.15	65667.37

Dynamic party additions. Assume the system is initialized with a domain size of $|\mathbb{H}| = 2^k$. When a new party joins the system, there are two possible scenarios based on the number of existing active parties:

- If the system is full, i.e., already has 2^k parties (including the dummy party), the current $\mathbb{H}$ cannot accommodate new evaluations. All parties must therefore re-execute the setup using a larger multiplicative subgroup $\mathbb{H}$.
- If the system is not full, the join process is extremely efficient. The new party first computes and submits its own hint with $O(N)$ complexity (Table 6). For existing parties, since their original hints depend solely on their own sk and the global $\mathbb{H}$ remains unchanged, they do not need to perform any operations. For the Preprocess algorithm, the system incrementally updates the parameters: it adds the new party's $\mathsf{hint}^{(1)}$ to vk, adds the new party's $\mathsf{hint}^{(5)}$ cross-terms to the corresponding existing entries in ak, and computes the new party's accumulated $\mathsf{hint}^{(5)}$ using the original raw hints provided by the existing parties (assuming the aggregator archives these original components). This incremental update requires at most $2N$ $\mathbb{G}_1$ additions (about 0.88s at $N = 2^{20}$) in total.

In summary, our scheme achieves highly practical dynamic party integration for the vast majority of scenarios.

10 DISCUSSION

Our work achieves a significant leap in aggregation efficiency by replacing the Plonk-style Hamming weight proofs for B and $\bar{B}$ in Dyna-hinTS with linear pairing checks. However, this efficiency gain inevitably introduces a trade-off in expressiveness: our scheme no longer natively supports non-trivial party weights or general access structures. The fundamental reason lies in the mathematical nature of our polynomial subset checks. Linear pairing checks can only verify inclusion relationships among the roots of polynomials (i.e., whether a party is present), but cannot verify non-zero numerical attributes without destroying the underlying algebraic structure. In contrast, a Plonk-style SNARK can easily support weighted structures by computing the inner product of B and a public weight vector w instead of $(1, 1, \ldots, 1)$. It can even support arbitrary access structures by replacing the inner product with custom circuit predicates. However, generating such a Plonk proof strictly requires the aggregator to perform $O(N \log N)$ operations over the global domain of size N, which inherently contradicts our goal of $O(n \cdot \mathrm{polylog}(n))$ aggregation efficiency. Consequently, designing a c-STS scheme that achieves both $O(n \cdot \mathrm{polylog}(n))$ aggregation efficiency and support for complex access structures remains a highly challenging open question.

The situation is even more challenging for STE. Currently, the only known method for constructing a practical STE is to leverage the compiler proposed in [27], which transforms a public linear verifiable STS into an STE. Therefore, the original scheme in [27] is inherently limited to unweighted threshold settings. Moreover, whether there exists any STE scheme supporting weighted structures—let alone c-STE schemes—remains unknown. We consider solving this an important and valuable direction for future research.

11 CONCLUSION

We presented the first practical constructions of the silent threshold signature scheme for dynamic committees (c-STS). For system size N and committee size n, we reduced the signature aggregation complexity from $O(N \log N)$ in prior work to $O(n \log^2 n)$ by introducing fully linear verification equations, together with a new committee representation and several polynomial-level optimizations. We further optimized the one-time silent setup phase, reducing the complexity of per-party setup from $O(N^2)$ to $O(N)$, while the signature size, verification key size, and verification time (with access to the epoch committee representation) of our scheme remained constant. Our Rust implementation confirms the practical impact of these improvements: with system parameters comparable to Ethereum, signature aggregation completes in about 0.15 seconds, and one-time setup becomes feasible even for systems with millions of parties.

Furthermore, we introduced the notion and practical construction of the silent threshold encryption scheme for dynamic committees (c-STE). This construction inherits the same efficiency and scalability benefits as our c-STS scheme. In conclusion, we demonstrate that silent threshold cryptography can be made highly practical for dynamic committee settings and deployed in real-world large-scale distributed systems.

ACKNOWLEDGMENTS

This work is supported by the National Natural Science Foundation of China (Grant No. 62272269), Scientific Research Innovation Capability Support Project for Young Faculty (Grant No. ZYGXQNJSKYCXNLZCXM-I21), and Primary Research and Development Plan of Jiangsu Province (Grant No. BE2023025).

REFERENCES

[1] [n. d.]. arkworks — arkworks.rs. https://arkworks.rs/. [Accessed 04-02-2026].
[2] [n. d.]. GitHub - dynaHintsRepo/dynaHints — github.com. https://github.com/dyna{H}ints{R}epo/dyna{H}ints.git. [Accessed 31-01-2026].
[3] [n. d.]. Proof of Stake Consensus - Polkadot Developer Docs — docs.polkadot.com. https://docs.polkadot.com/reference/polkadot-hub/consensus-and-security/pos-consensus/. [Accessed 05-02-2026].
[4] [n. d.]. Upgrading Ethereum | 2.9.5 Committees — eth2book.info. https://eth2book.info/capella/part2/building_blocks/committees/. [Accessed 05-02-2026].
[5] Inc. Algorand Foundation US. [n. d.]. Pure Proof-of-Stake - how Algorand's consensus mechanism works — algorand.co. https://algorand.co/technology/pure-proof-of-stake. [Accessed 05-02-2026].

[6] Thomas Attema and Ronald Cramer. 2020. Compressed-protocol theory and practical application to plug & play secure algorithmics. In *Annual International Cryptology Conference*. Springer, 513–543.

[7] Thomas Attema, Ronald Cramer, and Matthieu Rambaud. 2021. Compressed Σ-protocols for bilinear group arithmetic circuits and application to logarithmic transparent threshold signatures. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 526–556.

[8] Leemon Baird, Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. 2023. Threshold signatures in the multiverse. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1454–1470.

[9] Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Francois Garillot, Jonas Lindstrøm, Ben Riva, Arnab Roy, Mahdi Sedaghat, Alberto Sonnino, Pun Waiwitlikhit, and Joy Wang. 2024. Subset-optimized bls multi-signature with key aggregation. In *International Conference on Financial Cryptography and Data Security*. Springer, 188–205.

[10] Boaz Barak, Oded Goldreich, Rusell Impagliazzo, Steven Rudich, Amit Sahai, Salil Vadhan, and Ke Yang. 2001. On the (im) possibility of obfuscating programs. In *Annual international cryptology conference*. Springer, 1–18.

[11] Joshua Baron, Karim El Defrawy, Joshua Lampkins, and Rafail Ostrovsky. 2015. Communication-optimal proactive secret sharing for dynamic groups. In *International conference on applied cryptography and network security*. Springer, 23–41.

[12] Dan Boneh, Xavier Boyen, and Eu-Jin Goh. 2005. Hierarchical identity based encryption with constant size ciphertext. In *Annual international conference on the theory and applications of cryptographic techniques*. Springer, 440–456.

[13] Dan Boneh, Manu Drijvers, and Gregory Neven. 2018. Bls multi-signatures with public-key aggregation. *URL: https://crypto. stanford. edu/dabo/pubs/papers/BLSmultisig. html* (2018).

[14] Dan Boneh, Ben Lynn, and Hovav Shacham. 2001. Short signatures from the Weil pairing. In *International conference on the theory and application of cryptology and information security*. Springer, 514–532.

[15] Xavier Boyen. 2008. The uber-assumption family: A unified complexity framework for bilinear groups. In *International Conference on Pairing-Based Cryptography*. Springer, 39–56.

[16] Jeffrey Champion, Yao-Ching Hsieh, and David J Wu. 2025. Registered ABE and adaptively-secure broadcast encryption from succinct LWE. In *Annual International Cryptology Conference*. Springer, 3–34.

[17] Jeffrey Champion and David J Wu. 2024. Distributed broadcast encryption from lattices. In *Theory of Cryptography Conference*. Springer, 156–189.

[18] Tarun Chitra and Uthsav Chitra. 2019. Committee selection is more similar than you think: Evidence from avalanche and stellar. *arXiv preprint arXiv:1904.09839* (2019).

[19] Sourav Das, Philippe Camacho, Zhuolun Xiang, Javier Nieto, Benedikt Bünz, and Ling Ren. 2023. Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 356–370.

[20] Yvo Desmedt and Kirill Morozov. 2015. Parity check based redistribution of secret shares. In *2015 IEEE International Symposium on Information Theory (ISIT)*. IEEE, 959–963.

[21] Cody Freitag, Brent Waters, and David J Wu. 2023. How to use (plain) witness encryption: registered ABE, flexible broadcast, and more. In *Annual International Cryptology Conference*. Springer, 498–531.

[22] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. 2018. The algebraic group model and its applications. In *Annual International Cryptology Conference*. Springer, 33–62.

[23] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. 2019. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive* (2019).

[24] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. 2016. Candidate indistinguishability obfuscation and functional encryption for all circuits. *SIAM J. Comput.* 45, 3 (2016), 882–929.

[25] Sanjam Garg, Craig Gentry, Amit Sahai, and Brent Waters. 2013. Witness encryption and its applications. In *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*. 467–476.

[26] Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. 2024. hints: Threshold signatures with silent setup. In *2024 IEEE symposium on security and privacy (SP)*. IEEE, 3034–3052.

[27] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. 2024. Threshold encryption with silent setup. In *annual international cryptology conference*. Springer, 352–386.

[28] Timo Hanke, Mahnush Movahedi, and Dominic Williams. 2018. Dfinity technology overview series, consensus system. *arXiv preprint arXiv:1805.04548* (2018).

[29] Aniket Kate, Pratyay Mukherjee, Samipa Samanta, and Pratik Sarkar. 2025. DynahinTS: Silent Threshold Signatures for Dynamic Committees. *Cryptology ePrint Archive* (2025).

[30] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. 2010. Constant-size commitments to polynomials and their applications. In *International conference on the theory and application of cryptology and information security*. Springer, 177–194.

[31] Dimitris Kolonelos, Giulio Malavolta, and Hoeteck Wee. 2023. Distributed broadcast encryption from bilinear groups. In *International Conference on the Theory and Application of Cryptology and Information Security*. Springer, 407–441.

[32] Sai Krishna Deepak Maram, Fan Zhang, Lun Wang, Andrew Low, Yupeng Zhang, Ari Juels, and Dawn Song. 2019. CHURP: dynamic-committee proactive secret sharing. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2369–2386.

[33] Ueli Maurer. 2005. Abstract models of computation in cryptography. In *IMA International Conference on Cryptography and Coding*. Springer, 1–12.

[34] Silvio Micali, Leonid Reyzin, Georgios Vlachos, Riad S Wahby, and Nickolai Zeldovich. 2021. Compact certificates of collective knowledge. In *2021 IEEE Symposium on Security and Privacy (SP)*. IEEE, 626–641.

[35] Nicholas Pippenger. 1980. On the evaluation of powers and monomials. *SIAM J. Comput.* 9, 2 (1980), 230–250.

[36] Carla Ràfols and Arantxa Zapico. 2021. An algebraic framework for universal and updatable SNARKs. In *Annual International Cryptology Conference*. Springer, 774–804.

[37] Leonid Reyzin, Adam Smith, and Sophia Yakoubov. 2021. Turning HATE into LOVE: compact homomorphic ad hoc threshold encryption for scalable MPC. In *International Symposium on Cyber Security Cryptography and Machine Learning*. Springer, 361–378.

[38] David Schultz, Barbara Liskov, and Moses Liskov. 2010. MPSS: mobile proactive secret sharing. *ACM Transactions on Information and System Security (TISSEC)* 13, 4 (2010), 1–32.

[39] Hoeteck Wee and David J Wu. 2025. Unbounded distributed broadcast encryption and registered ABE from succinct LWE. In *Annual International Cryptology Conference*. Springer, 204–235.

[40] Lidong Zhou, Fred B Schneider, and Robbert Van Renesse. 2005. APSS: Proactive secret sharing in asynchronous systems. *ACM transactions on information and system security (TISSEC)* 8, 3 (2005), 259–286.

A CORRECTNESS AND UNFORGEABILITY DEFINITION OF C-STS

Following the idea of [26, 29], we still define the correctness and unforgeability of our c-STS scheme through two games between a challenger who controls the honest parties and an adversary $\mathcal{A}$ who may send arbitrary messages (for the hints and partial signatures) on behalf of the corrupted parties. However, we only focus on the number of parties that the adversary can control **within the epoch committee**: the protocol satisfies the security properties as long as this number is smaller than the threshold t. In contrast, our definition does not relate to the number of parties f that the adversary can control in the entire system, nor to the probability that the adversary fully controls a randomly sampled committee, which relates to the system-level parameter selection strategy and is logically independent of the cryptographic security of the protocol.

Definition A.1 (Correctness, revisited). The c-STS scheme satisfies correctness if, for any adversary $\mathcal{A}$ and any $N = \text{poly}(\lambda)$, the output of the correctness game defined below is 1 with probability 0.

1. The challenger runs $\text{CRS} \leftarrow \text{Setup}(1^\lambda, N)$ and gives CRS to $\mathcal{A}$.
2. $\mathcal{A}$ picks a subset A of corrupt parties such that $A \subseteq [N]$.
3. For all $i \in [N] \setminus A$, the public key and hint are sampled honestly by $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$ and $\text{hint}_i \leftarrow \text{HintGen}(\text{CRS}, sk, N, i)$.
4. For all $i \in A$, the adversary returns a public key pk_i and a hint hint_i to the challenger.
5. The challenger invokes the preprocessing as $(\text{ak}, \text{vk}) \leftarrow \text{Preprocess}(\text{CRS}, \{\text{hint}_i, pk_i\}_{i\in[N]})$ and the outputs are given to $\mathcal{A}$.
6. $\mathcal{A}$ picks an epoch e.

7. The challenger obtains the committee Com_e with its commitment by $B_e \leftarrow \mathsf{EpochRepr}(e)$ and gives it to $\mathcal{A}$.
8. $\mathcal{A}$ picks a message m, a threshold $1 \le t \le |\mathsf{Com}_e|$, and prepares the partial signatures $\{\sigma_i\}_{i\in S_1}$ for some subset of malicious parties $S_1 \subseteq A$. Let $S_1' \subseteq S_1$ be the subset of maliciously generated partial signatures that verify under PartVerify.
9. $\mathcal{A}$ may also request a subset of honest parties $S_2 \subseteq \mathsf{Com}_e \setminus A$ for partial signatures, which are returned by computing $\sigma_i \leftarrow \mathsf{Sign}(\mathsf{sk}_i, m, e)$.
10. The challenger computes the aggregated signature as $\sigma \leftarrow \mathsf{Aggr}(\mathsf{CRS}, \mathsf{ak}, m, \{\sigma_i\}_{i\in S}, t, e)$, where $S = (S_1' \cup S_2) \cap \mathsf{Com}_e$.
11. The output of the game is 1 if $|S| \ge t$ and $\mathsf{Verify}(\mathsf{vk}, m, \sigma, t, e) = 0$, else the output is 0.

Definition A.2 (Unforgeability, revisited). The c-STS scheme satisfies unforgeability if, for any PPT adversary $\mathcal{A}$ and any $N = \mathrm{poly}(\lambda)$, the output of the forgery game defined below is 1 with probability $\le \mathsf{negl}(\lambda)$.

1. The challenger runs $\mathsf{CRS} \leftarrow \mathsf{Setup}(1^\lambda, N)$ and gives CRS to $\mathcal{A}$.
2. $\mathcal{A}$ picks a subset A of corrupt parties such that $A \subseteq [N]$.
3. For all $i \in [N] \setminus A$, the public key and hint are sampled honestly by $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$ and $\mathsf{hint}_i \leftarrow \mathsf{HintGen}(\mathsf{CRS}, sk, N, i)$.
4. For all $i \in A$, the adversary returns a public key pk_i and a hint hint_i to the challenger.
5. The challenger invokes the preprocessing as $(\mathsf{ak}, \mathsf{vk}) \leftarrow \mathsf{Preprocess}(\mathsf{CRS}, \{\mathsf{hint}_i, pk_i\}_{i\in[N]})$ and the outputs are given to $\mathcal{A}$.
6. $\mathcal{A}$ picks an epoch e.
7. The challenger obtains the committee Com_e with its commitment by $B_e \leftarrow \mathsf{EpochRepr}(e)$ and gives it to $\mathcal{A}$.
8. $\mathcal{A}$ may make a partial signature query with a message m and an honest party $i \in [N] \setminus A$, and the query is returned by computing $\sigma_i \leftarrow \mathsf{Sign}(\mathsf{sk}_i, m, e)$. This is run as many times as desired by $\mathcal{A}$.
9. Finally, $\mathcal{A}$ shall output a challenge message m^* and a aggregated signature σ. Let S^* be the subset of honest parties queried by the adversary to sign m^*.
10. $\mathcal{A}$ wins the forgery game if there exist a threshold $|(S^* \cup A) \cap \mathsf{Com}_e| < t \le |\mathsf{Com}_e|$ such that $\mathsf{Verify}(\mathsf{vk}, m^*, \sigma, t, e) = 1$, in which case, the output of the game is 1. Otherwise, the output is 0.

B SECURITY ANALYSIS OF C-STS

Proof of correctness. For every registered public key $\mathsf{pk}_i \in \mathbb{G}_1$, let $x_i \in \mathbb{Z}_p$ be the unique scalar such that $\mathsf{pk}_i = [x_i]_1$.

First, consider any partial signature σ_i accepted by PartVerify. Writing $\mathsf{tag} = [\gamma]_2$, the verification equation implies

$$[x_i\gamma]_T = \mathsf{pk}_i \circ \mathsf{tag} = [1]_1 \circ \sigma_i.$$

Therefore, $\sigma_i = x_i \cdot \mathsf{tag}$, which shows that every accepted partial signature satisfies exactly the algebraic relation required by the aggregation algorithm, regardless of how it was generated.

Similarly, each equation in HintVerify uniquely determines the corresponding hint component. For example,

$$h_1 \circ [1]_2 = \mathsf{pk}_i \circ [L_i(\tau)]_2$$

implies $h_1 = [x_i L_i(\tau)]_1$, and the same argument applies to all the other components. Therefore, every accepted hint satisfies the same algebraic relations as the output of $\mathsf{HintGen}(\mathsf{CRS}, x_i, N, i)$. Notice that this argument only establishes the required algebraic relations; it does not rely on the accepted components having actually been generated by running the honest algorithm.

Let S be the set of accepted partial signatures after filtering and suppose that $|S| \ge t$. By Lemma 2.2 and the definitions of aPK, Q_Z, Q_x, and $\hat{Q}_x$, the values computed by Aggr satisfy

$$SK(X) \cdot B(X) = \mathsf{aSK} + Q_Z(X) \cdot Z(X) + Q_x(X) \cdot X,$$
$$Q_x(X) \cdot X = \hat{Q}_x(X),$$

where $\mathsf{aPK} = [\mathsf{aSK}]_1$. Moreover, since every accepted partial signature satisfies $\sigma_i = x_i \cdot \mathsf{tag}$, the aggregated signature satisfies $\sigma^* = \mathsf{aSK} \cdot \mathsf{tag}$. The remaining polynomials are constructed so that

$$X^{t+1} \cdot B(X) = \hat{B}(X)$$
$$B_e(X) \cdot B(X) = Z(X) \cdot \bar{B}(X)$$
$$B(X) - L_0(X) = (X-1)Q_0(X)$$

Evaluating these polynomial identities at $X = \tau$ yields exactly the six verification equations in Equation 6. Therefore,

$$\mathsf{Verify}(\mathsf{vk}, m, \sigma, t, e) = 1$$

and the correctness game outputs 1 with probability 0. □

Our proof of unforgeability relies on the Algebraic Group Model [22]. In this model, the adversary is considered algebraic, i.e., when the adversary outputs a group element $h_b \in \mathbb{G}_b$ (for any $b \in \{1, 2\}$), it must also output a vector v that satisfies $< v, v_b >= h_b$, where the vector v_b denotes all group elements from $\mathbb{G}_b$ that the adversary takes as input. We also assume that the following q-DLOG problem is hard for an algebraic adversary.

Definition B.1 (q-DLOG Assumption). For any positive integer q and algebraic adversary $\mathcal{A}$, it holds that

$$\Pr\left[x' = x \;\middle|\; x' = \mathcal{A}\begin{pmatrix} x \leftarrow \mathbb{Z}_p \\ [1]_1, [x]_1, \ldots, [x^q]_1, \\ [1]_2, [x]_2, \ldots, [x^q]_2 \end{pmatrix}\right] < \mathsf{negl}(\lambda).$$

Assuming the q-DLOG assumption, the following lemma [22] simplifies the security analysis in AGM.

Lemma B.2. *Let $[f_1(x_1, \ldots, x_l)], \ldots, [f_t(x_1, \ldots, x_l)]$ be a sequence of group elements (in either $\mathbb{G}_1$ or $\mathbb{G}_2$) given to an algebraic adversary $\mathcal{A}$ as input, where $x_1, \ldots, x_l \leftarrow \mathbb{Z}_p$. Let (g_1, g_2, h_1, h_2) be the output of $\mathcal{A}$. If it holds that $g_1 \circ h_1 = g_2 \circ h_2$, then with $1 - \mathsf{negl}(\lambda)$ probability, $\mathcal{A}$ must know the corresponding polynomials*

$$g_1(x_1, \ldots, x_l) = \sum_{i=1}^{t} \alpha_i \cdot f_i, h_1(x_1, \ldots, x_l) = \sum_{i=1}^{t} \beta_i \cdot f_i,$$
$$g_2(x_1, \ldots, x_l) = \sum_{i=1}^{t} \gamma_i \cdot f_i, h_2(x_1, \ldots, x_l) = \sum_{i=1}^{t} \theta_i \cdot f_i$$

such that

$$g_1(x_1, \ldots, x_l) \cdot h_1(x_1, \ldots, x_l) = g_2(x_1, \ldots, x_l) \cdot h_2(x_1, \ldots, x_l)$$

holds as a multivariate polynomial identity.

We prove the unforgeability of our scheme in the Algebraic Group Model below, starting with some preliminary lemmas.

Lemma B.3. *Suppose that at the end of the Forgery-Game defined in Definition A.2, the adversary $\mathcal{A}$ outputs a message and signature pair (m^*, σ) such that* $\mathsf{Verify}(\mathsf{vk}, m^*, \sigma, t, e) = 1$. *Then we can extract (multivariate) polynomials $B(X), \mathsf{aSK}(X,K), Q_Z(X,K), Q_x(X,K), \hat{Q}_x(X,K), \hat{B}(X), \bar{B}(X), Q_0(X)$ from $\mathcal{A}$ such that the following identities hold with probability $1 - \mathsf{negl}(\lambda)$. Here we use K to denote the variable set $\{\mathsf{sk}_i\}_{i\in[N]\setminus A}$, which is unknown to the adversary.*

$$
\begin{aligned}
SK(X,K)\cdot B(X) &= \mathsf{aSK}(X,K) + Q_Z(X,K)\cdot Z(X) + Q_x(X,K)\cdot X\\
Q_x(X,K)\cdot X &= \hat{Q}_x(X,K)\\
X^{t+1}\cdot B(X) &= \hat{B}(X)\\
B_e(X)\cdot B(X) &= Z(X)\cdot \bar{B}(X)\\
B(X) - L_0(X) &= Q_0(X)(X-1)
\end{aligned}
$$

Proof of Lemma B.3. Let $\sigma = (B, -\mathsf{aPK}, -Q_Z, -Q_x, \hat{Q}_x, \sigma^*, -\hat{B}, -\bar{B}, -Q_0)$. From $\mathsf{tag}\circ\mathsf{aPK} = [1]_1 \circ \sigma^*$ we have $\sigma^* = \mathsf{aSK}\cdot\mathsf{tag}$. Since the only group elements that $\mathcal{A}$ sees and are related to tag is $\{\mathsf{sk}_i\cdot\mathsf{tag}\}_{i\in S^*}$ and the signatures that are returned as outputs from the signing oracle, $\mathsf{aSK}(X,K)$ cannot depend on X and any sk_i that does not sign tag, i.e., we have $\mathsf{aSK}(K) = v_0 + \sum_{i\in S^*}\mathsf{sk}_i\cdot v_i$ for some $\{v_i\} \in \mathbb{Z}_p^{|S^*|+1}$.

Now we extract S from $B(X)$, and by Lemma B.4 we have $S \subseteq \mathsf{Com}_e$ and $|S| \geq t$. Let $\mathsf{aSK}' = \frac{1}{N+1}(\sum_{i\in S} B(\omega^i)\mathsf{sk}_i + 1)$. There should also exist an honestly sampled quotient polynomial $Q'_x(X,K), Q'_Z(X,K)$ whose terms that depend on K have degree $\leq N-1$ in terms of X and satisfy

$$SK(X,K)\cdot B(X) = \mathsf{aSK}' + Q'_Z(X,K)\cdot Z(X) + Q'_x(X,K)\cdot X.$$

Taking the difference shows that to forge a signature, $\mathcal{A}$ must compute polynomials $D_Z(X,K), D_x(X,K)$ that satisfy

$$\mathsf{aSK}' - \mathsf{aSK} = D_Z(X,K)\cdot Z(X) + D_x(X,K)\cdot X. \tag{7}$$

Since $t > |(S^*\cup A)\cap \mathsf{Com}_e|$ but $S \subseteq \mathsf{Com}_e$ and $|S| \geq t$, there must exist some $j \in S\setminus(S^*\cup A)$, and we have

$$\mathsf{aSK}' - \mathsf{aSK} = \frac{B(\omega^j)}{N+1}\mathsf{sk}_j + P(K\setminus\{\mathsf{sk}_j\}) + v_0$$

for some linear function P. Since $j \in S$, we have $c = \frac{B(\omega^j)}{N+1} \neq 0$.

Now we consider D_x. Since all inputs to $\mathcal{A}$ that depend on sk_i have degree $\leq N$ in terms of τ, the terms in $\hat{Q}_x(X)$ that depend on sk_i should have degree $\leq N$ in terms of X. Therefore, the terms in $Q_x(X) = \frac{\hat{Q}_x(X)}{X}$ that depend on sk_i should have degree $\leq N-1$ in terms of X, which implies the same for D_x. Taking the coefficient of the independent variable K_j in Equation 7 gives

$$c = d_Z(X)\cdot Z(X) + d_x(X)\cdot X,$$

where d_Z and d_x are the coefficients of K_j in D_Z and D_x, respectively, and $\deg(d_x) \leq N-1$. Since $Z(\omega^k) = 0$ and $\omega^k \neq 0$, we have $\omega^k\cdot d_x(\omega^k) = c$ for every $k \in [N+1]$. Therefore, the polynomial $R(X) = d_x(X)\cdot X - c$ vanishes at $\mathbb{H}$ and must be the zero polynomial since $\deg R \leq N$. However, at $X = 0$ we have $0 = R(0) = -c$, contradicting $c \neq 0$. Therefore, Equation 7 cannot hold, which concludes the proof. □

Since $\mathcal{A}$ is an algebraic adversary, these commitments must be a linear combination of the group elements that $\mathcal{A}$ takes as input, i.e., the outputs of $\mathcal{A}$ are multivariate polynomials depending on τ, K and $D = \{\mathsf{dlog}(\mathsf{tag}_i)\}_i$ (since $\{\mathsf{tag}_i\}_i$, as the outputs of the random oracle $\mathcal{H}$, have discrete logarithms that are uniformly random over $\mathbb{Z}_p$ and are algebraically independent of CRS and $\{\mathsf{pk}_i\}_{i\in[N]\setminus A}$). By Equation 6 and Lemma B.2, $\mathcal{A}$ must know polynomials $B(X), \mathsf{aSK}(X), Q_Z(X), Q_x(X), \hat{Q}_x(X), \hat{B}(X), \bar{B}(X), Q_0(X)$ which satisfy the identities

$$
\begin{aligned}
SK(X)\cdot B(X) &= \mathsf{aSK} + Q_Z(X)\cdot Z(X) + Q_x(X)\cdot X\\
Q_x(X)\cdot X &= \hat{Q}_x(X)\\
X^{t+1}\cdot B(X) &= \hat{B}(X)\\
B_e(X)\cdot B(X) &= Z(X)\cdot \bar{B}(X)\\
B(X) - L_0(X) &= Q_0(X)(X-1)
\end{aligned}
$$

We first argue that B only depends on τ:

- Suppose that B depends on some sk_i, then by $SK(X,K) = \sum_{i\in[N]\setminus A}\mathsf{sk}_i L_i(X) + f(X)$ for some polynomial $f(X)$, we know that $SK(X,K)\cdot B(X)$ must have a quadratic dependence on sk_i. However, the right-hand side of the first equation cannot have this dependence since the group elements that $\mathcal{A}$ takes as input are all linearly dependent on sk_i. Therefore, B is independent of any sk_i with probability $1 - \mathsf{negl}(\lambda)$.
- Suppose that B depends on some $d_i \in D$, then $\hat{B}$ must depend on the cross-term $d_i\cdot\tau^{t+1}$. However, such $\hat{B}$ cannot be linearly combined by $\mathcal{A}$ since none of its inputs depend on any $d_i\cdot\tau^k$ with $k > 0$. Therefore, B is independent of any d_i with probability $1 - \mathsf{negl}(\lambda)$.

Therefore, B only depends on τ, which implies the same for $\hat{B}, \bar{B}, Q_0$. Next, if Q_Z depends on some $d_i \in D$, then $Q_Z(X)\cdot Z(X)$ depends on the cross-term $d_i\cdot\tau^{N+1}$, which requires the same dependence for $Q_x(X)\cdot X$ since other parts of the first equation do not depend on any d_i. However, since none of $\mathcal{A}$'s inputs depend on any $d_i\cdot\tau^k$ with $k > 0$, $Q_x(X)\cdot X$ depends on at most $d_i\cdot\tau$, which leads to a contradiction. This shows that Q_Z cannot depend on D, which implies the same for Q_x and $\hat{Q}_x$. □

Lemma B.4. *Let $S = \{i \in [N] : B(\omega^i) \neq 0\}$, then $S \subseteq \mathsf{Com}_e$ and $|S| \geq t$.*

Proof of Lemma B.4. By $B(X) - L_0(X) = Q_0(X)(X-1)$ we have $B(1) = 1$, therefore $B(X)$ is not a zero polynomial. Suppose that $|S| < t$, then $B(X)$ must have $> N-t$ zeros on $\mathbb{H}$, which implies that $\deg(B) > N - t$. However, $B(X)$ must have degree $\leq N-t$ in terms of X since $X^{t+1}\cdot B(X) = \hat{B}(X)$ and $\hat{B}(X)$ must have degree $\leq N+1$ in terms of X. Finally, if there exists any $i \in S\setminus\mathsf{Com}_e$, then $B_e(\omega^i) \neq 0$ and $B_e(X)\cdot B(X) = Z(X)\cdot\bar{B}(X)$ does not hold at $X = \omega^i$, so we must have $S \subseteq \mathsf{Com}_e$. □

Proof of unforgeability. Suppose the adversary $\mathcal{A}$ wins the forgery game, then there exists a threshold $t > |(S^*\cup A)\cap\mathsf{Com}_e|$ such that $\mathsf{Verify}(\mathsf{vk}, m^*, \sigma, t, e) = 1$. By Lemma B.3 we have

$$SK(X,K)\cdot B(X) = \mathsf{aSK}(X,K) + Q_Z(X,K)\cdot Z(X) + Q_x(X,K)\cdot X.$$

C CORRECTNESS AND SEMANTIC SECURITY DEFINITION OF C-STE

Definition C.1 (Correctness). The c-STE scheme satisfies correctness if, for any adversary $\mathcal{A}$ and any $N = \text{poly}(\lambda)$, the output of the correctness game defined below is 1 with probability 0.

1. The challenger runs $\text{CRS} \leftarrow \text{Setup}(1^\lambda, N)$ and gives CRS to $\mathcal{A}$.
2. $\mathcal{A}$ picks a subset A of corrupt parties such that $A \subseteq [N]$.
3. For all $i \in [N] \setminus A$, the public key and hint are sampled honestly by $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$ and $\text{hint}_i \leftarrow \text{HintGen}(\text{CRS}, sk, N, i)$.
4. For all $i \in A$, the adversary returns a public key pk_i and a hint hint_i to the challenger.
5. The challenger invokes the preprocessing as $(\text{ak}, \text{ek}) \leftarrow \text{Preprocess}(\text{CRS}, \{\text{hint}_i, pk_i\}_{i\in[N]})$ and the outputs are given to $\mathcal{A}$.
6. $\mathcal{A}$ picks an epoch e.
7. The challenger obtains the committee Com_e with its commitment by $B_e \leftarrow \text{EpochRepr}(e)$ and gives it to $\mathcal{A}$.
8. $\mathcal{A}$ is given access to the partial decryption oracle of the honest parties, and it picks a message msg and a threshold $1 \le t \le |\text{Com}_e|$.
9. The honest ciphertext is generated by $\text{ct} \leftarrow \text{Enc}(\text{ek}, \text{msg}, t, e)$.
10. $\mathcal{A}$ prepares the partial decryptions $\{\sigma_i\}_{i\in S_1}$ for some subset of malicious parties $S_1 \subseteq A$. Let $S_1' \subseteq S_1$ be the subset of maliciously generated partial decryptions that verify under PartVerify.
11. $\mathcal{A}$ may also request a subset of honest parties $S_2 \subseteq \text{Com}_e \setminus A$ for partial decryptions, which are returned by computing $\sigma_i \leftarrow \text{PartDec}(\text{sk}_i, \text{ct}, e)$.
12. Let $S = (S_1' \cup S_2) \cap \text{Com}_e$. If $|S| \ge t$ (i.e., there are sufficiently many verified partial decryptions), the challenger computes the aggregated decryption as $\text{msg}' \leftarrow \text{DecAggr}(\text{CRS}, \text{ak}, \text{ct}, \{\sigma_i\}_{i\in S}, t, e)$.
13. The output of the game is 1 if $|S| \ge t$ and $\text{msg}' \neq \text{msg}$, else the output is 0.

Definition C.2 (Security). The c-STE scheme satisfies semantic security if, for any PPT adversary $\mathcal{A}$ and any $N = \text{poly}(\lambda)$, the output of the security game defined below is 1 with probability $\le \frac{1}{2} + \text{negl}(\lambda)$.

1. The challenger runs $\text{CRS} \leftarrow \text{Setup}(1^\lambda, N)$ and gives CRS to $\mathcal{A}$.
2. $\mathcal{A}$ picks a subset A of corrupt parties such that $A \subseteq [N]$.
3. For all $i \in [N] \setminus A$, the public key and hint are sampled honestly by $(\text{pk}, \text{sk}) \leftarrow \text{KeyGen}(1^\lambda)$ and $\text{hint}_i \leftarrow \text{HintGen}(\text{CRS}, sk, N, i)$.
4. For all $i \in A$, the adversary returns a public key pk_i and a hint hint_i to the challenger.
5. The challenger invokes the preprocessing as $(\text{ak}, \text{ek}) \leftarrow \text{Preprocess}(\text{CRS}, \{\text{hint}_i, pk_i\}_{i\in[N]})$ and the outputs are given to $\mathcal{A}$.
6. $\mathcal{A}$ picks an epoch e.
7. The challenger obtains the committee Com_e with its commitment by $B_e \leftarrow \text{EpochRepr}(e)$ and gives it to $\mathcal{A}$.
8. $\mathcal{A}$ picks messages $\text{msg}_0, \text{msg}_1 \in \mathbb{G}_T$ and a threshold $1 \le t \le |\text{Com}_e|$.
9. The challenger picks a bit $b \leftarrow \{0, 1\}$ and generates a ciphertext $\text{ct} \leftarrow \text{Enc}(\text{ek}, \text{msg}_b, t, e)$.
10. $\mathcal{A}$ outputs a bit b' and wins the semantic security game if $t > |A \cap \text{Com}_e|$ and $b' = b$, in which case the output of the game is 1. Otherwise, the output is 0.

D SECURITY ANALYSIS OF C-STE

Proof of correctness. By the correctness of the underlying c-STS aggregation algorithm, the witness $\boldsymbol{w}$ reconstructed from any set of at least t accepted partial decryptions satisfies

$$\boldsymbol{A} \circ \boldsymbol{w} = \boldsymbol{b}.$$

Let the honestly generated ciphertext be

$$\text{ct} = (u, \boldsymbol{s}^\top \boldsymbol{A}, \boldsymbol{s}^\top \boldsymbol{b} + \text{msg}).$$

Then the output of DecAggr is

$$\begin{aligned}\text{msg}^* &= ct_3 - \boldsymbol{ct}_2 \circ \boldsymbol{w} \\ &= \boldsymbol{s}^\top \boldsymbol{b} + \text{msg} - (\boldsymbol{s}^\top \boldsymbol{A}) \circ \boldsymbol{w} \\ &= \boldsymbol{s}^\top \boldsymbol{b} + \text{msg} - \boldsymbol{s}^\top (\boldsymbol{A} \circ \boldsymbol{w}) \\ &= \text{msg}.\end{aligned}$$

Therefore, whenever at least t valid partial decryptions are provided, DecAggr recovers msg with probability 1. □

Our proof for the security of our scheme relies on the Generic Group Model [12, 15, 33] and its master theorem.

Theorem D.1 (Master Theorem [12, 15, 33]). *Let $\boldsymbol{L}_1 \in \mathbb{Z}_p[X_1, \ldots, X_n]^{v_1}$, $\boldsymbol{L}_2 \in \mathbb{Z}_p[X_1, \ldots, X_n]^{v_2}$, $\boldsymbol{L}_T \in \mathbb{Z}_p[X_1, \ldots, X_n]^{v_T}$ be three lists of n-variate polynomials over $\mathbb{Z}_p$ of maximum degree $d_{L_1}, d_{L_2}, d_{L_T}$, respectively. Let $f \in \mathbb{Z}_p[X_1, \ldots, X_n]$ be an n-variate polynomial of degree d_f. Denote $d = \max\{d_{L_1}, d_{L_2}, d_{L_T}\}$ and $v = v_1 + v_2 + v_T$. If f is independent of $(\boldsymbol{L}_1, \boldsymbol{L}_2, \boldsymbol{L}_T)$, then for any generic adversary $\mathcal{A}$ that makes at most q group oracle queries we have*

$$\left|\Pr\left[\mathcal{A}\left(\begin{array}{c} p, \mathsf{h}_1[\boldsymbol{L}_1(\boldsymbol{x})], \\ \mathsf{h}_2[\boldsymbol{L}_2(\boldsymbol{x})], \mathsf{h}_T[\boldsymbol{L}_T(\boldsymbol{x}), f(\boldsymbol{x})] \end{array}\right) = 1\right]\right.$$

$$\left. - \Pr\left[\mathcal{A}\left(\begin{array}{c} p, \mathsf{h}_1[\boldsymbol{L}_1(\boldsymbol{x})], \\ \mathsf{h}_2[\boldsymbol{L}_2(\boldsymbol{x})], \mathsf{h}_T[\boldsymbol{L}_T(\boldsymbol{x}), r] \end{array}\right) = 1\right]\right| \le \frac{(q + v + 2)^2 \cdot d}{2p},$$

where $\mathsf{h}_1, \mathsf{h}_2, \mathsf{h}_T$ denote the corresponding handles, and the probabilities are taken over the choices of $\boldsymbol{x} \leftarrow_\$ (\mathbb{Z}_p)^n$ and $r \leftarrow_\$ \mathbb{Z}_p$.

Definition D.2 (Independence). For lists $\boldsymbol{L} = (\boldsymbol{L}_1, \boldsymbol{L}_2, \boldsymbol{L}_T)$, let the completion

$$\begin{aligned}\boldsymbol{C}(\boldsymbol{L}) &= \{\boldsymbol{L}_1 \otimes \boldsymbol{L}_2\} \cup \boldsymbol{L}_T \\ &= \{g_1(X_1, \ldots, X_n), \ldots, g_D(X_1, \ldots, X_n)\}\end{aligned}$$

are of size D. We say that f is dependent on $\boldsymbol{L}$ if there exist coefficients $\kappa_i \in \mathbb{Z}_p$ such that

$$f(X_1, \ldots, X_n) = \sum_{i=1}^{D} \kappa_i \cdot g_i(X_1, \ldots, X_n).$$

Otherwise, we say that f is independent of $\boldsymbol{L}$.

Proof of security. Our proof follows the technique of [27]: To show that it is computationally hard for a generic adversary that learns the elements specified by our construction to distinguish

between $[s_6 \cdot L_0(\tau)]_T$ and a random element $[r]_T$, it is necessary to apply Theorem D.1, and we work in the symbolic group model to argue that $S_6 \cdot L_0(X)$ is symbolically independent of the lists of (symbolic) elements that arise in the security game of our construction (and after the completion).

Without loss of generality, we assume that all corrupted parties submitted valid hints (corrupted parties with invalid hints are removed from the system during preprocessing and are ignored thereafter; hence, we denote $H = [N] \setminus A$ and $H' = H \cup \{0\}$. Let $B_e(X) = \prod_{i \in \mathsf{Com}'_e}(X - \omega^i)$.

Game$_0$: This is the semantic security game of Definition C.2. Since we assume that $\mathcal{H}(\cdot)$ is a random oracle, we write $\mathsf{tag} = [\gamma]_2$ where $\gamma \in \mathbb{Z}_p$ remains unknown and is hard to compute.

Game$_1$: This is the same as **Game**$_0$ but we simplify the view of $\mathcal{A}$: the challenger gives $\{[\mathsf{sk}_i \tau^j]_1\}_{i \in H, j \in [N]}$ to $\mathcal{A}$. In contrast, $\{\mathsf{hint}_i\}_{i \in H}, \mathsf{ak}, \mathsf{ek}$ are no longer provided.

Game$_2$: Compared to **Game**$_1$, we change the ciphertext to $\mathsf{ct}' = (u, \boldsymbol{s}^\top \boldsymbol{A}, [r]_T + \mathsf{msg}_b)$, where $u, r \leftarrow_\$ \mathbb{Z}_p$ are sampled uniformly at random by the challenger, and the challenger queries the random oracle $\mathcal{H}(\cdot)$ with (u, e) to obtain $[\gamma]_2 = \mathcal{H}(u, e)$. We have $\Pr[\mathbf{Game}_2 = 1] = \frac{1}{2}$ since it is obvious that ct' leaks no information about msg_b.

We denote the formal variables as $\boldsymbol{Y} = (X, \boldsymbol{K}, \Gamma, \boldsymbol{S})$, where $\boldsymbol{K} = (K_i)_{i \in H}, \boldsymbol{S} = (S_i)_{i=1}^6$, and $\boldsymbol{Y}$ corresponds to the elements $(\tau, (\mathsf{sk}_i)_{i \in H}, \gamma, (s_i)_{i=1}^6)$ that are hidden to $\mathcal{A}$.

Since $L_0(X) = \frac{1}{N+1} \cdot \frac{X^{N+1}-1}{X-1} = \frac{1}{N+1} \cdot \sum_{i=0}^{N} X^i$, we can represent the handles $\boldsymbol{L}_1, \boldsymbol{L}_2, \boldsymbol{L}_T, f$ in **Game**$_1$ as follows:

$$\boldsymbol{L}_1(\boldsymbol{Y}) = \Big(1, X, \ldots, X^{N+1}, \{K_i, K_i X, \ldots, K_i X^N\}_{i \in H}, \overbrace{S_3}^{\boldsymbol{s}^\top \boldsymbol{a}_6}, \overbrace{S_1\big(L_0(X) + \sum_{i \in H} K_i L_i(X) + \sum_{i \in A} \mathsf{sk}_i L_i(X)\big) + S_4 X^{t+1} + S_5 B_e(X) + S_6}^{\boldsymbol{s}^\top \boldsymbol{a}_1}\Big)$$

$$\boldsymbol{L}_2(\boldsymbol{Y}) = \big(1, X, \ldots, X^{N+1}, \Gamma, \overbrace{S_1 + \Gamma S_3}^{\boldsymbol{s}^\top \boldsymbol{a}_2}, \overbrace{S_1 X^{N+1} - S_1}^{\boldsymbol{s}^\top \boldsymbol{a}_3}, \overbrace{S_1 X + S_2 X}^{\boldsymbol{s}^\top \boldsymbol{a}_4}, \overbrace{S_2}^{\boldsymbol{s}^\top \boldsymbol{a}_5}, \overbrace{S_4}^{\boldsymbol{s}^\top \boldsymbol{a}_7}, \overbrace{S_5 X^{N+1} - S_5}^{\boldsymbol{s}^\top \boldsymbol{a}_8}, \overbrace{S_6 X - S_6}^{\boldsymbol{s}^\top \boldsymbol{a}_9}\big)$$

$$\boldsymbol{L}_T(\boldsymbol{Y}) = \Big(1, \sum_{i=0}^{N} X^i\Big)$$

$$f(\boldsymbol{Y}) = S_6 L_0(X) = \frac{S_6}{N+1} \cdot \sum_{i=0}^{N} X^i$$

Now we want to prove that f is independent of $\boldsymbol{L} = (\boldsymbol{L}_1, \boldsymbol{L}_2, \boldsymbol{L}_T)$ and therefore we can apply Theorem D.1. Assume that f is dependent on $\boldsymbol{L}$, which is equivalent to the existence of $\kappa_i \in \mathbb{Z}_p$ such that $S_6 L_0(X) = \sum_{i=1}^{D} \kappa_i \cdot g_i(\boldsymbol{Y})$, where $\boldsymbol{C}(\boldsymbol{L}) = \{g_1(\boldsymbol{Y}), \ldots, g_D(\boldsymbol{Y})\}$ is the completion of $\boldsymbol{L}$. We divide the terms in this completion into three types:

- Terms that do not involve $\boldsymbol{S}$, e.g., $X^3 \cdot \Gamma$.
- Terms that involve $\boldsymbol{S}$ on both sides, e.g., $S_2 \cdot S_3$.
- Terms that involve $\boldsymbol{S}$ on exactly one side, e.g., $X^2 \cdot (S_1 + \Gamma S_3)$.

Since the first two types cannot symbolically play any role in $S_6 L_0(X) = \sum_{i=1}^{D} \kappa_i \cdot g_i(\boldsymbol{Y})$, we are only left with the third type of terms, which shows that $S_6 L_0(X) = \sum_{i=1}^{9} \boldsymbol{S}^\top \boldsymbol{a}_i g_i(X, \boldsymbol{K}, \Gamma) = \boldsymbol{S}^\top \boldsymbol{A} \cdot \boldsymbol{g}(X, \boldsymbol{K}, \Gamma)$. Moreover, by $S_6 L_0(X) = \boldsymbol{S}^\top \boldsymbol{b}$, it symbolically holds that $\boldsymbol{A} \cdot \boldsymbol{g}(X, \boldsymbol{K}, \Gamma) = \boldsymbol{b}$. We prove that this cannot happen if $t > |A \cap \mathsf{Com}_e|$ in the following lemma.

Lemma D.3. *Let $\boldsymbol{L}_1 = (1, \{X^j\}_{j \in [N+1]}, \{K_i X^j\}_{i \in H, j \in [N]'})$, $\boldsymbol{L}_2 = (1, \{X^j\}_{j \in [N+1]}, \Gamma)$, and matrix $\boldsymbol{A} \in \mathbb{Z}_p[X, (K_i)_{i \in H}, \Gamma]^{6 \times 9}$ defined as*

$$\boldsymbol{A} = \begin{pmatrix} F_1(\boldsymbol{Y}) & 1 & F_2(\boldsymbol{Y}) & X & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & X & 1 & 0 & 0 & 0 & 0 \\ 0 & \Gamma & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ X^{t+1} & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ F_4(\boldsymbol{Y}) & 0 & 0 & 0 & 0 & 0 & 0 & F_2(\boldsymbol{Y}) & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & F_3(\boldsymbol{Y}) \end{pmatrix}$$

where

$$F_1(\boldsymbol{Y}) = L_0(X) + \sum_{i \in H} K_i L_i(X) + \sum_{i \in A} \mathsf{sk}_i L_i(X)$$
$$F_2(\boldsymbol{Y}) = X^{N+1} - 1$$
$$F_3(\boldsymbol{Y}) = X - 1$$
$$F_4(\boldsymbol{Y}) = \prod_{i \in \mathsf{Com}'_e} (X - \omega^i)$$

and $\boldsymbol{b} = (0, 0, 0, 0, 0, L_0(X))^\top$. If $t > |A \cap \mathsf{Com}_e|$, there does not symbolically exist any non-zero $\boldsymbol{g} \in \mathbb{Z}_p[X, (K_i)_{i \in H}, \Gamma]^9$ (where g_1, g_6 are from $\boldsymbol{L}_2$ and $g_2, g_3, g_4, g_5, g_7, g_8, g_9$ are from $\boldsymbol{L}_1$ respectively) such that $\boldsymbol{A} \cdot \boldsymbol{g} = \boldsymbol{b}$.

Proof. Assume that there is a $\boldsymbol{g} = (g_i)_{i=1}^9$ such that $\boldsymbol{A} \cdot \boldsymbol{g} = \boldsymbol{b}$. Then we have

$$g_1 F_1(\boldsymbol{Y}) + g_2 + g_3(X^{N+1} - 1) + g_4 X = 0 \tag{8}$$
$$g_4 X + g_5 = 0 \tag{9}$$
$$g_2 \Gamma + g_6 = 0 \tag{10}$$
$$g_1 X^{t+1} + g_7 = 0 \tag{11}$$
$$g_1 F_4(\boldsymbol{Y}) + g_8(X^{N+1} - 1) = 0 \tag{12}$$
$$g_1 + g_9(X - 1) = L_0(X). \tag{13}$$

Moreover, we have

$$g_k = \begin{cases} \sum_{j=0}^{N+1} \alpha_j^{(k)} X^j + \delta^{(k)} \Gamma, & k = 1, 6 \\ \sum_{j=0}^{N+1} \alpha_j^{(k)} X^j + \sum_{i \in H} \sum_{j=0}^{N} \beta_{i,j}^{(k)} K_i X^j, & k = 2, 3, 4, 5, 7, 8, 9 \end{cases}$$

We analyze the structure of each g_k below. For g_1:

- Since Γ does not appear in Equation 8, we have $\delta^{(1)} = 0$ and
$$g_1 = g_1(X) = \sum_{j=0}^{N+1} \alpha_j^{(1)} X^j.$$
- From Equation 11 we have
$$\sum_{j=0}^{N+1} \alpha_j^{(1)} X^j \cdot X^{t+1} = \sum_{j=0}^{N+1} -\alpha_j^{(7)} X^j + \sum_{i \in H} \sum_{j=0}^{N} -\beta_{i,j}^{(7)} K_i X^j$$

so $\beta_{i,j}^{(7)} = 0$ for all i, j and $\alpha_j^{(1)} = 0$ for $j > N - t$, which shows that $g_1(X) = \sum_{j=0}^{N-t} \alpha_j^{(1)} X^j$.

- From Equation 12 we have

$$\sum_{j=0}^{N-t} \alpha_j^{(1)} X^j \cdot \prod_{i \in \mathsf{Com}'_e} (X - \omega^i) = \left(\sum_{j=0}^{N+1} -\alpha_j^{(8)} X^j + \sum_{i \in H} \sum_{j=0}^{N} -\beta_{i,j}^{(8)} K_i X^j \right) (X^{N+1} - 1)$$

so $\beta_{i,j}^{(8)} = 0$ for all i, j and $g_8 = g_8(X)$. Therefore, from $g_1(X) \cdot \prod_{i \in \mathsf{Com}'_e} (X - \omega^i) = g_8(X)(X^{N+1} - 1)$ we have $g_1(\omega^i) = 0$ for $i \notin \mathsf{Com}'_e$.

- From Equation 13 we have

$$\sum_{j=0}^{N-t} \alpha_j^{(1)} X^j - L_0(X) = \left(\sum_{j=0}^{N+1} -\alpha_j^{(9)} X^j + \sum_{i \in H} \sum_{j=0}^{N} -\beta_{i,j}^{(9)} K_i X^j \right) (X - 1)$$

so $\beta_{i,j}^{(9)} = 0$ for all i, j and $g_9 = g_9(X)$. Therefore, from $g_1(X) - L_0(X) = g_9(X)(X - 1)$ we have $g_1(1) = 1$.

For g_2: From Equation 10 we have

$$\left(\sum_{j=0}^{N+1} \alpha_j^{(2)} X^j + \sum_{i \in H} \sum_{j=0}^{N} \beta_{i,j}^{(2)} K_i X^j \right) \Gamma = \sum_{j=0}^{N+1} -\alpha_j^{(6)} X^j + (-\delta^{(6)}) \Gamma$$

so $\alpha_j^{(2)}, \alpha_j^{(6)}, \beta_{i,j}^{(2)} = 0$ for all i, j except for $\alpha_0^{(2)}$, which shows that $g_6 = \delta^{(6)} \Gamma$ and $g_2 = \alpha_0^{(2)}$ is a constant.

For g_4: From Equation 9 we have

$$\left(\sum_{j=0}^{N+1} \alpha_j^{(4)} X^j + \sum_{i \in H} \sum_{j=0}^{N} \beta_{i,j}^{(4)} K_i X^j \right) X = \sum_{j=0}^{N+1} \alpha_j^{(5)} X^j + \sum_{i \in H} \sum_{j=0}^{N} \beta_{i,j}^{(5)} K_i X^j \tag{14}$$

so $\alpha_0^{(5)} = 0$ and $\beta_{i,0}^{(5)} = 0$ for all i.

Now we use Lemma 2.2 in Equation 8. Since $g_1(X) = \sum_{i=0}^{N+1} b_i L_i(X)$ with $g_1(1) = 1$ and $g_1(\omega^i) = 0$ for $i \notin \mathsf{Com}'_e$, we have

$$\begin{aligned} &\sum_{i \in H \cap \mathsf{Com}_e} K_i b_i + \sum_{i \in A \cap \mathsf{Com}_e} \mathsf{sk}_i b_i + 1 + Q_x(X) X + Q_Z(X)(X^{N+1} - 1) \\ =&\alpha_0^{(2)} + \left(\sum_{j=0}^{N+1} \alpha_j^{(3)} X^j + \sum_{i \in H} \sum_{j=0}^{N} \beta_{i,j}^{(3)} K_i X^j \right) (X^{N+1} - 1) \\ +&\left(\sum_{j=0}^{N+1} -\alpha_j^{(5)} X^j + \sum_{i \in H} \sum_{j=0}^{N} -\beta_{i,j}^{(5)} K_i X^j \right) \end{aligned}$$

where $g_4 X$ is replaced with g_5 according to Equation 14. So $\beta_{i,j}^{(3)} = 0$ for all i, j, and furthermore $\beta_{i,j}^{(5)} = 0$ for all i, j, which shows that $b_i = 0$ for $i \in H \cap \mathsf{Com}_e$. Since we already have $b_0 = 1$ and $b_i = 0$ for $i \notin \mathsf{Com}'_e$, it must hold that $\deg(g_1) \geq N - |A \cap \mathsf{Com}_e|$. However, $g_1(X) = \sum_{j=0}^{N-t} \alpha_j^{(1)} X^j$ shows that $\deg(g_1) \leq N - t$, hence $|A \cap \mathsf{Com}_e| \geq t$, which contradicts our assumption. □

Therefore, we conclude that f is independent of $\boldsymbol{L}_1, \boldsymbol{L}_2, \boldsymbol{L}_T$. Since $v = \mathrm{poly}(\lambda), d = \mathrm{poly}(\lambda), p = \Theta(2^\lambda)$ and for every PPT $\mathcal{A}$ we have $q = \mathrm{poly}(\lambda)$, Theorem D.1 provides

$$|\Pr[\mathbf{Game}_1 = 1] - \Pr[\mathbf{Game}_2 = 1]| \leq \frac{(q + v + 2)^2 \cdot d}{2p} = \mathrm{negl}(\lambda).$$

Therefore, by $\Pr[\mathbf{Game}_2 = 1] = \frac{1}{2}$ we have $\Pr[\mathbf{Game}_1 = 1] = \frac{1}{2} + \mathrm{negl}(\lambda)$. Finally, the following lemma demonstrates the equivalence between $\mathbf{Game}_0$ and $\mathbf{Game}_1$:

Lemma D.4 (Equivalence of the honest-hint representation). *For every $i \in [N] \setminus A$, the group elements contained in pk_i and hint_i span exactly the same $\mathbb{Z}_p$-linear space as*

$$\{[K_i X^j]_1\}_{j=0}^{N}.$$

Moreover, the linear transformations in both directions are publicly computable.

Proof. For $j \neq i$, define

$$P_{i,j}(X) = \frac{L_i(X) L_j(X)}{Z(X)}.$$

Each $P_{i,j}$ has degree at most $N - 1$. For every $k \neq i$,

$$P_{i,j}(\omega^k) = 0 \quad \text{whenever } k \neq j,$$

whereas

$$P_{i,j}(\omega^j) = \frac{1}{N+1} \cdot \frac{\omega^i}{\omega^j - \omega^i} \neq 0.$$

Consequently, the evaluation matrix of $\{P_{i,j}\}_{j \neq i}$ on the N points $\{\omega^k : k \neq i\}$ is diagonal with non-zero diagonal entries. The polynomials $\{P_{i,j}\}_{j \neq i}$ are therefore linearly independent. Since the space of polynomials of degree at most $N - 1$ has dimension N, they form a basis of that space.

The fifth components of hint_i are precisely

$$\{[K_i P_{i,j}(X)]_1\}_{j \neq i}.$$

They therefore span

$$\{[K_i X^j]_1\}_{j=0}^{N-1}.$$

Furthermore, $L_i(X)$ has degree exactly N and a non-zero leading coefficient. Hence, adding

$$[K_i L_i(X)]_1$$

extends the preceding basis to a basis of all polynomials of degree at most N. It follows that pk_i and hint_i can be used to compute every element $[K_i X^j]_1$ for $0 \leq j \leq N$.

Conversely, every polynomial occurring in hint_i has degree at most N. Therefore, all components of pk_i and hint_i can be computed as public linear combinations of $\{[K_i X^j]_1\}_{j=0}^{N}$. This proves the claimed equivalence. □

Lemma D.4 shows that the replacement between $\mathbf{Game}_0$ and $\mathbf{Game}_1$ does not change $\mathcal{A}$'s view, hence $\Pr[\mathbf{Game}_0 = 1] = \Pr[\mathbf{Game}_1 = 1]$, which concludes the proof. □

E OPEN SCIENCE

To support the reproducibility of our work and to assist the program committee in evaluating the paper's core contributions, we have provided all necessary artifacts through the following anonymous website:

https://github.com/Gorachya/c-STS-STE

Our submission includes the source code for implementing our schemes, and our evaluation results can be obtained by following the commands in `README.md`.

F ETHICAL CONSIDERATIONS

This work focuses on the theoretical design and practical implementation of advanced cryptographic protocols, specifically silent threshold signature (STS) and encryption (STE) schemes. Our research does not involve any human subjects, the collection or processing of user data, nor the exploitation or analysis of vulnerabilities in deployed real-world systems. Consequently, we consider this work to have no related ethical concerns.